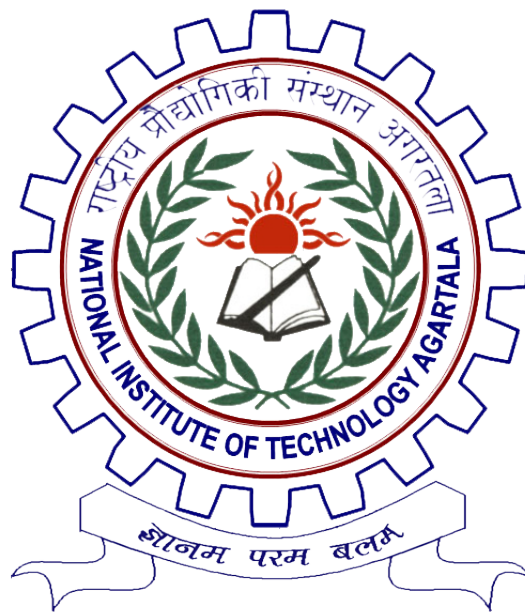


CREDIT CARD FRAUD DETECTION USING MACHINE LEARNING



Sahil Dey(20UCS008)
Shubhadeep Nath(20UCS001)
Sourajita Chanda(20UCS041)
Tithi Majumder(20UCS131)

**COMPUTER SCIENCE & ENGINEERING DEPARTMENT
NATIONAL INSTITUTE OF TECHNOLOGY, AGARTALA**

**INDIA-799046
MAY, 2024**

CREDIT CARD FRAUD DETECTION USING MACHINE LEARNING

*Report submitted to
National Institute of Technology, Agartala
for the award of the degree
of
Bachelor of Technology*

*by
Sahil Dey(20UCS008)
Shubhadeep Nath(20UCS001)
Sourajita Chanda(20UCS041)
Tithi Majumder(20UCS131)*

Under the Guidance of

*Dr. Kunal Chakma
Assistant Professor*



**COMPUTER SCIENCE & ENGINEERING DEPARTMENT
NATIONAL INSTITUTE OF TECHNOLOGY AGARTALA
MAY, 2024**

Dedicated To

To our Project Supervisor **Dr. Kunal Chakma, Associate Professor, CSED, NIT Agartala** for sharing his valuable knowledge, and encouragement and for showing confidence in us all the time. Each of the faculties of the department to contribute in our development as a professional and help us to achieve this goal. To all those people who have somehow contributed to the creation of this project and who have supported us.

“If I have seen further than others, it is by standing upon the shoulders of giants.”

- Isaac Newton

REPORT APPROVAL FOR B.TECH

This report entitled “*CREDIT CARD FRAUD DETECTION USING MACHINE LEARNING*”, by Sahil Dey, Shubhadeep Nath, Sourajita Chanda, Tithi Majumder is approved in partial fulfilment of the requirements for the award of *Bachelor of Technology* in *Computer Science & Engineering*.

Dr. Kunal Chakma
(Project Supervisor)
Assistant Professor
Computer Science and Engineering Department
NIT, Agartala

Dr. Suman Deb
(Head of the Department)
Assistant Professor
Computer Science and Engineering Department
NIT Agartala

Date:_____

Place:NIT, Agartala

DECLARATION

We declare that the work presented in this report titled “*CREDIT CARD FRAUD DETECTION USING MACHINE LEARNING*”, submitted to the Computer Science and Engineering Department, National Institute of Technology, Agartala, for the award of the ***Bachelor of Technology*** degree in ***Computer Science & Engineering***, represents our ideas in our own words and where others’ ideas or words have been included, we have adequately cited and referenced the original sources. We also declare that we have adhered to all principles of academic honesty and integrity and have not misrepresented or fabricated or falsified any idea/data/fact/source in our submission. We understand that any violation of the above will be a cause for disciplinary action by the Institute and can also evoke penal action from the sources which have thus not been properly cited or from whom proper permission has not been taken when needed.

MAY, 2024

Agartala

Sahil Dey(20UCS008)

Shubhadeep Nath
(20UCS001)

Tithi Majumder (20UCS131)

Sourajita Chanda
(20UCS041)

CERTIFICATE

It is certified that the work contained in the report titled “*CREDIT CARD FRAUD DETECTION USING MACHINE LEARNING*”, by Sahil Dey, Shubhadeep Nath, Sourajita Chanda, Tithi Majumder has been carried out under my supervision. This work has not been submitted elsewhere for a degree.

Dr. Kunal Chakma
Assistant Professor
Computer Science and Engineering Department
NIT Agartala

Acknowledgement

We would like to take this opportunity to express our deep sense of gratitude to all who helped us directly or indirectly during this thesis work. Firstly, we would like to thank our supervisor, **Dr. Kunal Chakma**, for being a great mentor and the best adviser we could ever have. His advice, encouragement and criticism are sources of innovative ideas, inspiration and causes behind the successful completion of this report. His confidence shown to us was the biggest source of inspiration for us. It has been a privilege working with him for the past 6 months. We are highly obliged to all the faculty members of Computer Science and Engineering Department for their support and encouragement. We also thank **Prof.(Dr.) S. K. Patra**, Director, NIT Agartala and **Dr. Suman Deb**, H.O.D, CSED for providing excellent computing and other facilities without which this work could not achieve its quality goal.

- **Shubhadeep Nath(20UCS001)**

- **Sahil Dey(20UCS008)**

- **Sourajita Chanda(20UCS041)**

- **Tithi Majumder(20UCS131)**

List of Figures

3.1	Class Distribution	14
3.2	Average Transaction Time Comparison: Fraudulent vs. Non-Fraudulent Classes	15
3.3	Average Amount for Fraud & Non-Fraud Cases	16
3.4	Transaction Patterns Across Hours	18
3.5	Non-Fraudulent Transactions Patterns Across Hours	19
3.6	Fraudulent Transactions Patterns Across Hours	20
3.7	Average Transaction Amounts Across Hours	21
3.8	Average Fraud and Non-Fraud Transactional Amounts Across Hours	22
3.9	Histogram Visualization of Numerical Features in Credit Card Data	23
3.10	Correlation Heatmap	25
3.11	Pair-Plot	27
4.1	System Flowchart	31
4.2	Neural Network Illustration	35

5.1	Confusion Matrix for Logistic Regressor	45
5.2	Confusion Matrix for SVM	46
5.3	Confusion Matrix for Random Forest	47
5.4	Confusion Matrix for Neural Network	48
5.5	Confusion Matrix for KNN	49
5.6	Confusion Matrix for Decision Tree	50
5.7	Accuracy Barplot for different models	55

List of Tables

1	ROC AUC Metrics	43
2	K-Folds for Logistic Regression	52
3	K-Folds for SVM	52
4	K-Folds for Random Forest	53
5	K-Folds for Neural Network	53
6	K-Folds for KNN	54
7	K-Folds for Decision Tree	54
8	Accuracy Details	55

Abstract

The project investigates the use of machine learning techniques, specifically Random Forest, Support Vector Machines (SVM), Logistic Regression, Neural Network, KNN & Decision Tree to develop an advanced credit card fraud detection system. The model gains the capacity to differentiate between authentic and fraudulent transactions through the analysis of past transaction data, which increases its flexibility in response to changing fraud strategies. Transactional data serves as the basis for analysis, and each algorithm is meticulously honed and trained to spot trends that might indicate fraudulent behaviour. The model's performance is analysed in detail, with an emphasis on accuracy, precision, recall, and F1 score metrics. The study's conclusions shed light on the benefits and drawbacks of each approach, helping to choose the ideal model for practical use. These findings validate the viability of using machine learning to fraud detection and offer valuable details on algorithmic nuances that will facilitate the creation of an operational credit card fraud detection system at peak efficiency.

Contents

Acknowledgement	viii
Abstract	xii
1 INTRODUCTION	1
1.1 Motivation	1
1.2 Objective	2
1.3 Advantages	2
1.4 Contributions of the Work	3
1.5 Organisation of the thesis	4
2 LITERATURE REVIEW	5
2.1 Literature Review	5
3 DATASET	8

3.1	Problem Statement	8
3.2	Description of Dataset	8
3.3	Data Preprocessing	9
3.4	Handling Class Imbalance	10
3.5	Exploratory Data Analysis	10
3.5.1	Display Dataframe	11
3.5.2	Dataframe Info	11
3.5.3	Dataframe Missing Value Summary	12
3.5.4	Numerical Column Summary	13
3.5.5	Transaction Class Distribution	13
3.5.6	Time Comparison	15
3.5.7	Median Transaction Amounts	16
3.5.8	Hourly Transaction Distribution	17
3.5.9	Hourly Average Transactions Amounts	21
3.5.10	Histograms of Numerical Features	23
3.5.11	Correlation Heatmap	25
3.5.12	Pair Plot Analysis	27
3.5.13	Insights from EDA	29
3.6	Data Splitting	29
4	PROPOSED MODEL	31
4.1	Description	31

4.2	Algorithms	32
4.2.1	Logistic Regression	32
4.2.2	Support Vector Machine	33
4.2.3	Random Forest	33
4.2.4	Neural Networks :	34
4.2.5	K-Nearest Neighbour Algorithm(KNN) :	35
4.2.6	Decision Tree :	36
4.3	Experimental Setup	36
4.3.1	Import Libraries	36
4.3.2	Model Evaluation	37
5	RESULTS AND ANALYSIS	41
5.1	Classification Metrics	41
5.1.1	Confusion Matrix	41
5.2	Analysis of Results	42
5.2.1	ROC AUC Metrics	43
5.2.2	Confusion Matrix Analysis	45
5.2.3	Model Tuning	51
5.2.4	Analysis of Model Tuning	51
5.2.5	Accuracy Result Analysis	54
5.2.6	Model Comparison	55
5.2.7	Statistical Significance Test	56

5.2.8	Statistical Significance Test Result Analysis	56
6	CONCLUSION & FUTURE SCOPES	57
6.1	Consclusion	57
6.2	Future Scopes	58
7	Biographical sketch	60

CHAPTER 1

INTRODUCTION

With the increase in digital transactions, credit card theft has developed as a permanent and sophisticated danger to the financial system. With the rise of online commerce and electronics payment systems, fraudsters are constantly inventing new ways to exploit vulnerabilities in a transactional environment, resulting in significant financial losses. Like traditional payment methods migrate to the digital arena, efficient and adaptable credit card fraud detection systems become critical. Machine learning became a common term when it is about identifying fraudulent cases in real time because the sheer volume of complex data collected every day. Machine learning techniques help in identifying abnormalities in big dataset, because of the presence of advanced machine learning architectures it is possible to recognize fraudulent transactions more precisely and efficiently. So this research on credit card fraud detection makes a valuable contribution to the creation of advanced and flexible solution. By enabling financial institutions to outperform criminals proactively, these innovations foster a dependable and safe digital financial environment.

1.1 Motivation

Motivation for this project came from the rapidly increasing threat of defrauding crimes in the e-commerce and digital payments sector. The rise of online transactions has been accompanied

exponentially with fraudulent activities through credit card payments which is making the financial industry more vulnerable than ever. To diminish these deceitful activities, to gain customer trust and satisfaction and to counter these fraudulent schemes, strong and innovative solutions are being developed using machine learning algorithms. These attempts solely aim to ensure security of electronic and digital payment systems helping in ensuring customer confidence over the digital payments sector.

1.2 Objective

Objective of this project is to implement modern technologies and analytical ways to effectively and precisely recognize and avert defrauding schemes. This approach aims on reducing monetary losses, gaining customer trust and ensuring the integrity of the electronic payments sector as well as the financial industry.

1.3 Advantages

The model on fraud detection has the following advantages:

- **Prevents Financial Losses:** Reduces fiscal losses by recognizing and averting defrauding transactions.
- **Gaining Trust of Customers:** Gains and carry on the trust of the users, So that they are not afraid of using credit cards.
- **Obedience with Directives:** Make sure that the regulatory standards related to defrauding activities minimization are being adhered seriously.
- **Reducing The Risk:** Reducing all the risks and managing those risks associated with deceitful activities.
- **Security of Customer's Data:** Mitigating the danger of data breaches by safeguarding sensitive information and customer's metadata.
- **Measures to Adapt with the system:** Different architectures and algorithms are used to avert the fraudulent schemes.
- **Efficiency:** Instantaneous engagement which minimizes the human intervention and promises quick response time.

- **Diminishing the False Positives:** Ensures minimizing legitimate transactions being mistakenly flagged as fraudulent.
- **Making Decisions using trends:** Analysis of data to identify trends and vulnerabilities for informed decision-making.

1.4 Contributions of the Work

This work contributes towards the ongoing and evolving efforts to identify, prevent, and mitigate fraudulent activities. This involves the application of data analytics and machine learning techniques to detect anomalies, unauthorized access, and fraudulent patterns. This work further contributes to:

- **Real-time Monitoring:** Continuously monitoring credit card transactions in real-time to identify suspicious activities promptly.
- **Pattern Recognition:** Utilizing data analytics to recognize patterns indicative of fraud, including unusual spending behavior, geographic anomalies, and transaction irregularities.
- **Predictive Modeling:** Employing predictive modeling and machine learning algorithms to anticipate potential fraud based on historical data and emerging patterns.
- **Behavioral Analysis:** Analyzing user behavior to establish a baseline and detect deviations that might indicate fraudulent activities.
- **Collaborative Solutions:** Encouraging collaboration among financial institutions, payment processors, and law enforcement agencies to share information and enhance the collective ability to combat fraud.
- **User Education:** Including initiatives to educate users about safe practices, security measures, and how to recognize and report suspicious activities.
- **Continuous Improvement:** Embracing a culture of continuous improvement by regularly updating and refining fraud detection systems to address new challenges and vulnerabilities.

1.5 Organisation of the thesis

The rest of the thesis is organised into the following chapters:

- **Chapter 2:** The literature review is discussed in this chapter.
- **Chapter 3:** This chapter focuses on the dataset description, preprocessing of the given data, handling the imbalances of the classes, Exploratory Data Analysis(EDA),Data Splitting,
- **Chapter 4:** This chapter focuses on the description of the proposed models for the given dataset, the algorithms that are implemented, the experimental setup that is used and evaluation of the models
- **Chapter 5:** This chapter deals with the analysis and comparison between the results of the different proposed models in the previous chapter
- **Chapter 6:**The conclusion of the report is described in this chapter & also described the future scopes of the project.
- **Chapter 7:**All the references & the inspirations that we have taken and countered for this project are mentioned in this chapter.

CHAPTER 2

LITERATURE REVIEW

2.1 Literature Review

Numerous publications on Credit Card Fraud Detection System were looked through during the literature review process. A few of these articles examined and suggested various techniques for detecting fraud using various angles. Some, however, emphasised that, as opposed to depending solely on heuristic-based techniques, fraud detection in credit card transactions is a supervised learning classification job.

Clifton Phua and Vincent Lee et. al. [2] it has been revealed that techniques employed in this domain include data mining applications, automated fraud detection, adversarial detection. Suman et. al. [3], Research Scholar, GJUS&T at Hisar HCE presented techniques like Supervised and Unsupervised Learning for credit card fraud detection. Even though these methods and algorithms fetched an unexpected success in some areas, they failed to provide a permanent and consistent solution to fraud detection. Vaishnavi Nath et. al. [4] proposed an approach approach first categorizes cardholders into groups based on their transaction amounts using range partitioning. Then, it applies a sliding-window analysis to each group's transaction data, extracting features like maximum, minimum, and average transaction amounts, along with time elapsed. This method reveals insights into cardholders' spending behaviors, aiding in targeted strategies

for marketing, risk assessment, or fraud detection based on each group's distinct characteristics. Debachudamani Prusti and Santhnu Kumar Rath et. al. [5] designed an application with applied machine learning approaches such as Decision tree (DT), k-nearest algorithm (kNN), Extreme learning machine (ELM), Multilayer perceptron (MLP) and support vector machine (SVM) to detect the accuracy in fraud identification. They proposed a model by hybridizing the DT, SVM and kNN techniques. They used two web-based protocols such as simple object access protocol (SOAP) and Representational state transfer (REST) for efficient exchange of data across multiple heterogeneous platforms. They compared five machine learning algorithm results based on accuracy metric. SVM performed better than other algorithms

Neural networks, inspired by the structure and function of the human brain, have emerged as powerful tools for fraud detection. They excel at learning complex patterns and relationships in data, making them well-suited for detecting fraudulent transactions. Researchers have explored various types of neural networks, including feedforward neural networks, convolutional neural networks (CNNs), recurrent neural networks (RNNs), and more recently, deep learning architectures such as deep belief networks (DBNs) and autoencoders.

Ryman-Tubb and Paul Krause et. al. [6] suggested an approach which centers on training neural networks with historical credit card transaction data to discern fraud patterns. Subsequently, they advocate for the extraction of transparent rules from these networks. These rules are then deployed in live Fraud Management Systems, addressing concerns about the complexity of neural networks in critical business operations. By providing interpretable rules, this method streamlines the workload of fraud analysts while ensuring adaptability to changing fraud tactics through regular rule extraction. Overall, it offers a practical solution for integrating neural network-based fraud detection systems into crucial business processes. Ghosh, S., Reilly et. al. [7] Credit card fraud detection with a neural-network. In: Proceedings of the Twenty-Seventh Hawaii International Conference on System Sciences, 1994, outlines the deployment and performance evaluation of a neural network-based fraud detection system for credit card transactions. Trained on a large dataset containing labeled transactions, the system was tested on a separate holdout dataset spanning a two-month period. It demonstrated superior fraud detection capabilities compared to rule-based methods, detecting significantly more fraud accounts while reducing false positives by a factor of 20. Installed on an IBM 3090 at Mellon Bank, the system is currently operational, showcasing its effectiveness in enhancing fraud detection in the bank's credit card portfolio.

Neural networks have emerged as powerful tools for credit card fraud detection, offering the

potential to improve detection accuracy and efficiency compared to traditional methods. While significant advancements have been made, challenges such as dataset imbalance and model interpretability remain areas of active research. By addressing these challenges and leveraging emerging technologies, neural network-based fraud detection systems can continue to evolve and provide robust protection against fraudulent activities in the financial sector.

CHAPTER 3

DATASET

3.1 Problem Statement

The challenge in detecting fraud in a credit card transaction, with the necessary knowledge and modeling based on transaction records, is to verify whether the developed model can accurately identify and determine whether a new transaction is fraudulent or not.

3.2 Description of Dataset

A sizable collection of transactions, each with a variety of attributes that reveal information about the transaction, make up the dataset for the credit card fraud detection project. Usually, the dataset include labels that indicate whether or not each transaction is authentic or fraudulent. To carry out the study, the dataset was gathered from the Kaggle website.

Here's a brief description of the key components in the given dataset:

1. Transaction Features:

- **Time:** Timestamp indicating when the transaction occurred.
- **Amount:** The monetary value of the transaction.
- **V1, V2, ..., V28:** Anonymized features representing various aspects of the transaction. These features are often the result of dimensionality reduction techniques like Principal

Component Analysis (PCA) to protect sensitive information.

2. **Class Label:**

- **Class (Fraud or Legitimate):** A binary label indicating whether the transaction is fraudulent (Class 1) or legitimate (Class 0)

3. **Imbalanced Class Distribution:**

- Credit card fraud datasets are often imbalanced, with a majority of transactions being legitimate. The imbalance ratio between fraudulent and legitimate transactions can be significant

4. **Real-time vs Historical Data:**

- The dataset may represent historical transaction data, allowing for retrospective analysis. In some cases, datasets might be designed to simulate real-time processing

3.3 Data Preprocessing

Data preprocessing involves the systematic preparation and transformation of raw data to improve its quality and suitability for analysis or machine learning applications. This process includes addressing missing values, handling duplicates and outliers, scaling numerical features, encoding categorical variables, and performing other adjustments to ensure the data is ready for effective exploration, modeling, and interpretation.

1. **Handling Duplicate Values:** Repeated entries in a dataset can provide biased analysis and model training. Identifying and eliminating duplicate rows is critical for integrity and avoiding distorted findings.
2. **Handling Null Values:** Void or missing values can cause problems in studies and model training, perhaps leading in biased or incomplete results. Addressing null values with imputation or elimination results in a more full and trustworthy dataset.
3. **Handling Outliers:** Machine learning models and statistical summaries can be greatly impacted by outliers, which can result in erroneous conclusions and forecasts. It is essential to recognise and deal with outliers in a way that minimises their effects and guarantees reliable and accurate modelling or analysis.
4. **Data Scaling:** In machine learning, discrepancies in feature scaling can lead to biased

model training and erroneous predictions. Standardising or normalising numerical characteristics through data scaling guarantees that all features are given equal consideration fairly during model training, hence strengthening model performance and interpretability.

3.4 Handling Class Imbalance

To address the challenge of imbalanced classes, the "Imbalanced Learn" Python library is utilized. This library is specifically designed to tackle situations where there is a skewed distribution of classes in the target variable of the dataset. Such imbalance can lead to biases in the training data, especially towards the majority class. Implementing Imbalanced Learn involves various techniques such as oversampling the minority class, undersampling the majority class, and generating synthetic samples using SMOTE (Synthetic Minority Oversampling Technique). By integrating Imbalanced Learn, we can enhance the model's performance on minority classes, leading to more accurate predictions, particularly in scenarios where class imbalance poses a significant concern.

3.5 Exploratory Data Analysis

- **Data Overview:** We conducted exploratory data analysis (EDA) to understand the structure and properties of our datasets.
- **Distribution Analysis:** Studying data distributions allows for understanding the central tendencies and variability of important variables, laying the groundwork for future research.
- **Correlation Assessment:** Correlations between variables were investigated, revealing potential linkages and dependencies that are critical for making educated decisions.
- **Outlier Identification:** Outlier detection procedures have been implemented to discover and rectify abnormalities, hence maintaining data integrity and dependability.
- **Pattern Recognition:** Used Python tools and modules to create visualisations that helped identify patterns and trends in the datasets.

- **Informative Visuals:** Plots, charts, and graphs were created to present complicated data linkages in a clear and succinct manner, allowing stakeholders to understand them more easily.
- **Foundation for Analysis:** EDA was the fundamental element, giving a thorough grasp of the data ecosystem and directing later analytical efforts.

3.5.1 Display Dataframe

The `credit_card.head()` method displays the initial rows of the Pandas Data frame 'credit_card'. It serves as a quick overview, presenting the first 5 rows by default, to gain insights into the dataset's structure and content.

This method is specifically associated with Pandas DataFrames :

1. Overview:

- `credit_card` is assumed to be a Pandas DataFrame representing a dataset related to credit card transactions (as suggested by the name).
- The `head()` method is used to display the first few rows of the DataFrame.

2. Syntax:

- The `head()` method is called on a DataFrame, and it returns the first 5 rows (by default) of the DataFrame.

3. Purpose:

- It's primarily used for a quick exploration of the dataset to get a sense of its structure, contents, and the format of the data.
- Viewing the first few rows is helpful in understanding the column names, data types, and example values in each column.

3.5.2 Dataframe Info

The `credit_card.info()` method in Python, specifically when applied to a Pandas DataFrame named `credit_card`, provides concise information about the DataFrame's structure.

1. Overview:

- The `credit_card.info()` method in Python, applied to a Pandas DataFrame named `credit_card`, provides a concise summary of the DataFrame's structure.
- It offers insights into the number of entries, data types, non-null counts, and memory usage, aiding in a quick assessment of the dataset's characteristics.

2. Purpose:

- Get a quick understanding of the DataFrame structure without viewing the complete dataset.
- Describe the data types in each column to aid understanding of the data.
- Get an understanding into the DataFrame's index, including the kind and range of index values.
- Streamline data cleaning processes by identifying columns that have missing values or require type conversions.

3.5.3 Dataframe Missing Value Summary

The `credit_card.isna().sum()` method in Python, is used to calculate the number of missing values (NaN or null values) in each column of the `credit_card` Pandas DataFrame.

1. Overview:

- The expression `credit_card.isna().sum()` is used in Python with Pandas to generate a summary of missing values within each column of the `credit_card` DataFrame. It provides a quick overview of the count of missing values, aiding in the identification of areas in the dataset that require attention or imputation.
- It offers insights into the number of entries, data types, non-null counts, and memory usage, aiding in a quick assessment of the dataset's characteristics.

2. Purpose:

- It efficiently counts the number of missing values in each column of the DataFrame.
- Offers a quick overview of where data might be incomplete, helping to identify columns with missing information.
- Essential for data cleaning tasks, enabling prioritization of columns for imputation or removal of missing values.

3.5.4 Numerical Column Summary

The `credit_card.describe()` method in Python, when applied to a Pandas DataFrame named `credit_card`, provides a statistical summary of the numerical columns within the DataFrame.

1. Overview:

- The `credit_card.describe()` method in Python, when applied to a Pandas DataFrame such as `credit_card`, generates a statistical summary of the numerical columns within the DataFrame.
- The output provides key measures such as the mean, standard deviation, and quartiles, offering a quick and informative overview of the distribution and central tendencies of the data.

2. Purpose:

- Describes the central location of the data, providing the mean (average) value.
- Offers insights into the distribution by presenting values at different percentiles.
- Assists in making informed decisions about data preprocessing, cleaning, and understanding the overall characteristics of the dataset.
- Indicates the spread or variability of the data through measures like standard deviation.

3.5.5 Transaction Class Distribution

The fig 3.1 visualizes the distribution of the 'Class' column in a DataFrame, presumably the `credit_card` by implementing the `px.pir()` function from the `plotly.express` library to create a pie chart.

1. Overview:

- It uses Plotly Express to create a pie chart. The 'Class' column from the `credit_card` DataFrame is used for the pie slices, and additional parameters specify the chart's title, color scheme, and the size of the central hole, creating a donut chart.

Distribution of Class Column

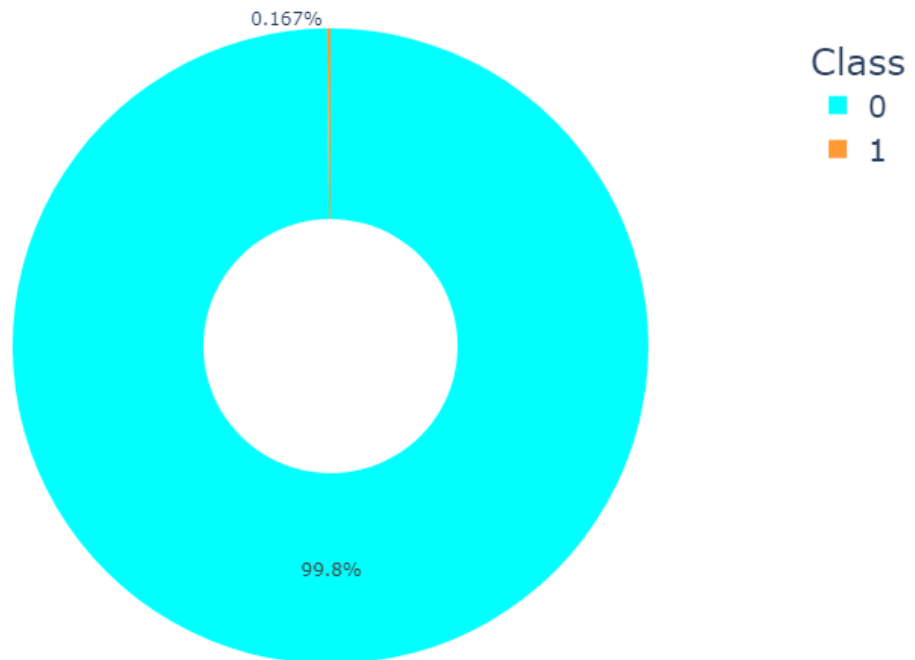


Figure 3.1: Class Distribution

- Modifies the layout settings of the chart. It specifies that the legend should be shown with a title 'Class', and the legend items should have constant sizing. The plot background color is set to white.

2. Purpose:

- The purpose of this code is to visually represent the distribution of classes in the 'Class' column of the `credit_card` DataFrame using a pie chart.
- In this context, it provides an intuitive overview of the distribution of the 'Class' column, where classes are represented by different colored segments in the chart.

3.5.6 Time Comparison

Fraudulent vs Non-Fraudulent Cases The fig 3.2 here visualizes the distribution of time among the fraud and non-fraud transactions in the form of bar graph in the DataFrame, presumably the `credit_card` by implementing the `px.bar()` function from the `plotly.express` library to create a bar chart.



Figure 3.2: Average Transaction Time Comparison: Fraudulent vs. Non-Fraudulent Classes

1. Overview:

- The code calculates the average time for each class (fraudulent and non-fraudulent) in a credit card dataset.
- It then uses Plotly Express to create a bar chart, with the x-axis representing the classes, the y-axis representing the average time, and text labels displaying the precise average time on each bar.

2. Purpose:

- The chart aims to visually compare and contrast the average times associated with fraudulent and non-fraudulent credit card transactions.
- It provides a quick and intuitive summary of the temporal aspects of these two classes, allowing for easy identification of any patterns or differences in average transaction times.

3.5.7 Median Transaction Amounts

Fraudulent vs Non-Fraudulent Cases The fig 3.3 here visualizes the distribution of total mounts among the fraud and non-fraud transactions in the form of bar graph in the DataFrame, presumably the `credit_card` by implementing the `px.bar()` function from `plotly.express` library to create a bar chart.

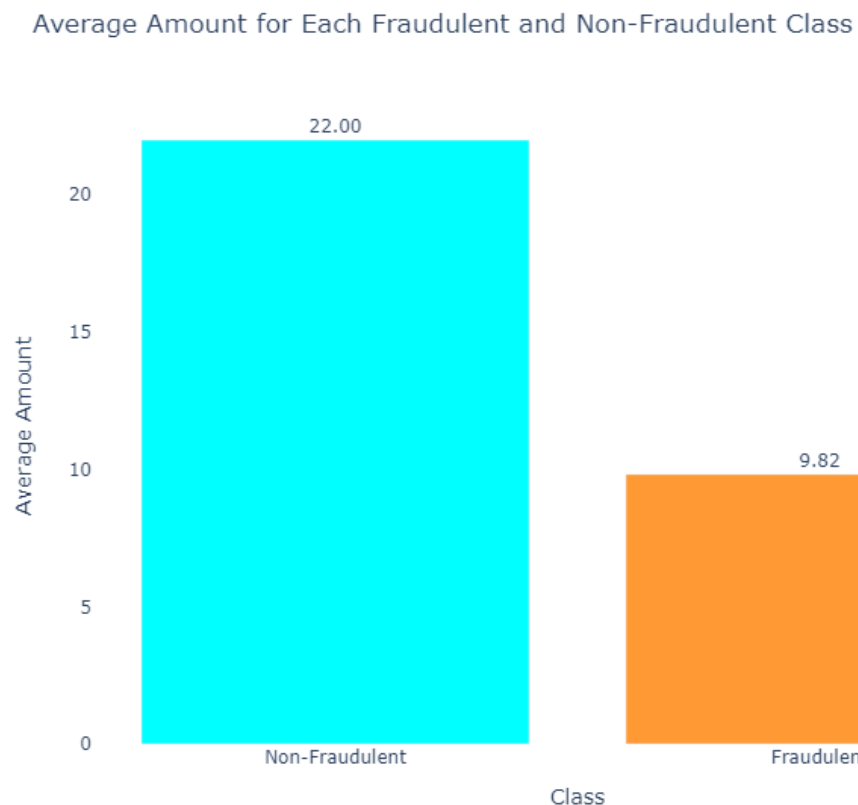


Figure 3.3: Average Amount for Fraud & Non-Fraud Cases

1. Overview:

- The code groups the credit card dataset by the 'Class' column and calculates the median transaction amount for each class.
- It utilizes Plotly Express to create a bar chart, where the x-axis represents the classes, the y-axis represents the median transaction amount, and text labels display the precise median amounts on each bar.

2. Purpose:

- The bar chart aims to provide a quick and intuitive comparison of median transaction amounts between fraudulent and non-fraudulent credit card transactions.
- The visualization facilitates the identification of potential differences in transaction amounts associated with different classes, aiding in exploratory data analysis.
- By using medians, the chart is less sensitive to extreme values and provides a robust measure of central tendency for transaction amounts in each class.

3.5.8 Hourly Transaction Distribution

1. **Hourly Transaction Distribution** The fig 3.4 here visualizes the number of total transactions with respect to the hour of the day in the form of grouped bar graph in the DataFrame, presumably the `credit_card` by implementing the `px.bar()` function from the `plotly.express` library to create a grouped bar chart.

(a) Overview:

- The code uses `hourly_transaction`, presumably a DataFrame, to represent transaction counts grouped by hour.
- The chart provides insights into hourly transaction patterns, aiding in the identification of trends and variations throughout the day.

(b) Purpose:

- The chart visually represents the distribution of transaction counts across different hours of the day.
- The chart offers insights into the temporal dynamics of transactions, aiding in understanding daily patterns.

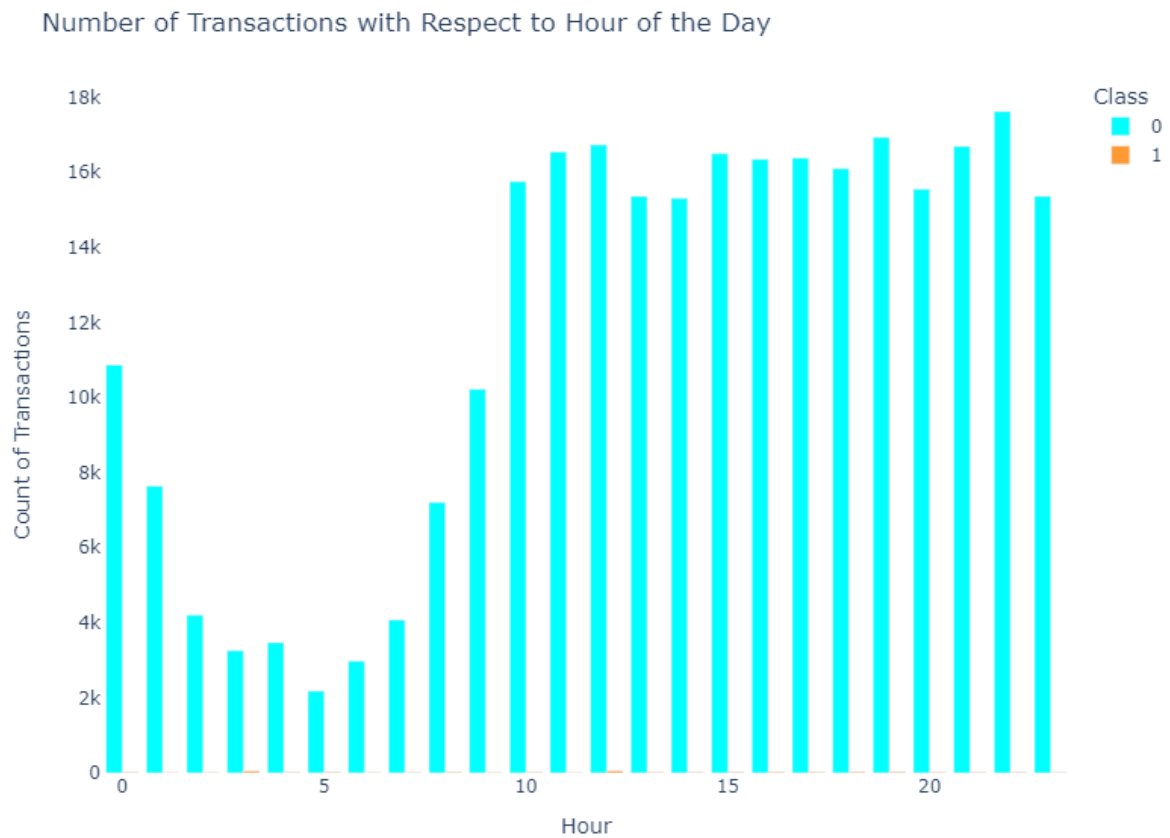


Figure 3.4: Transaction Patterns Across Hours

2. **Distribution of Non-Fraudulent Transactions Across Hours** The fig 3.5 here visualizes the number of non-fraudulent transactions with respect to the hour of the day in the form of grouped bar graph in the DataFrame, presumably the `credit_card` by implementing the `px.bar()` function from the `plotly.express` library to create a grouped bar chart.

(a) Overview:

- Extracts the count of non-fraudulent transactions (assuming 0 represents non-fraudulent) from the `hourly_transaction` DataFrame and resets the index, creating a new DataFrame `class_0_df`.
- Uses Plotly Express to create a bar chart specifically for non-fraudulent transactions.

(b) Purpose:

Distribution of Non-Fraud Transactions in Each Hour

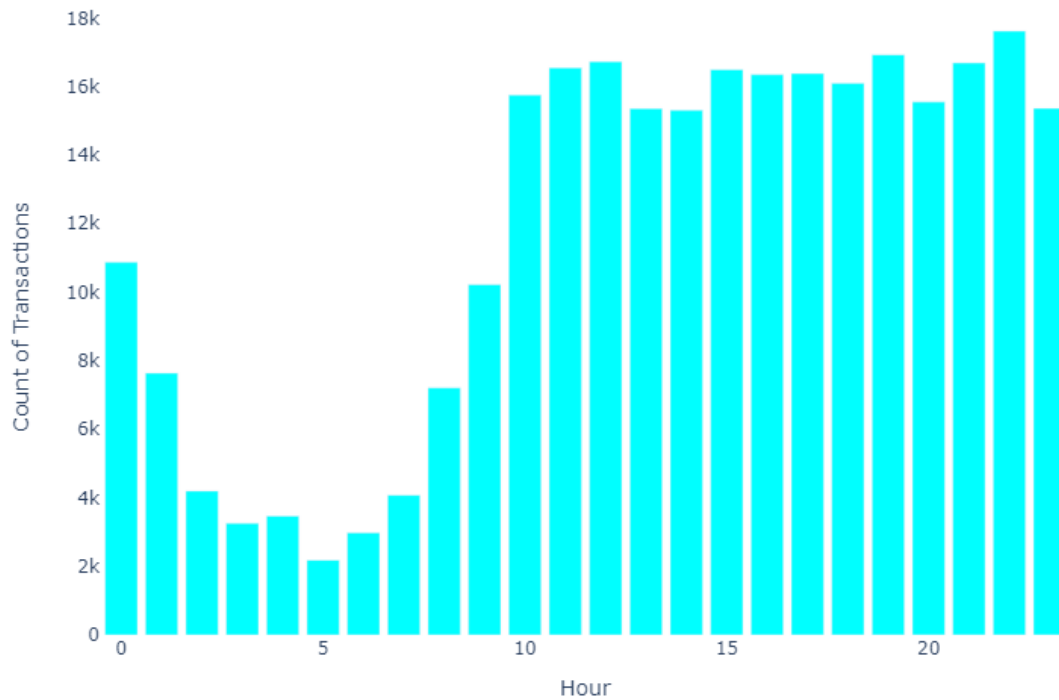


Figure 3.5: Non-Fraudulent Transactions Patterns Across Hours

- The code aims to provide a focused visualization of the distribution of non-fraudulent transactions across different hours of the day.
- The bar chart helps in understanding the patterns and variations in non-fraudulent transaction counts, with a clear emphasis on visual clarity through color and layout adjustments.

3. **Distribution of Fraudulent Transactions Across Hours** The fig 3.6 here visualizes the number of fraudulent transactions with respect to the hour of the day in the form of grouped bar graph in the DataFrame, presumably the `credit_card` by implementing the `px.bar()` function from the `plotly.express` library to create a grouped bar chart.

(a) Overview:

- Extracts data for fraudulent transactions (Class 1) from `hourly_transaction`, creating `class_1_df`.

Distribution of Fraud Transactions in Each Hour

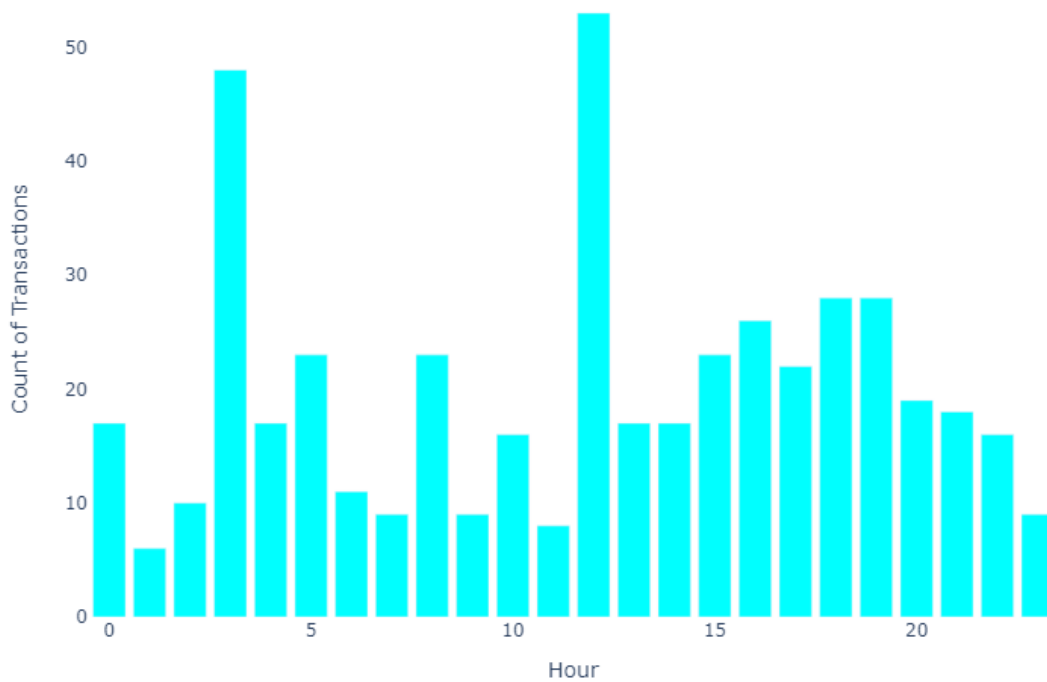


Figure 3.6: Fraudulent Transactions Patterns Across Hours

- Uses Plotly Express (`px.bar`) to generate a bar chart for fraudulent transactions.

(b) Purpose:

- The code aims to visually represent and compare the distribution of fraudulent transactions across different hours of the day.
- The bar chart provides a clear and focused visualization of hourly transaction patterns specifically for fraudulent transactions, with distinctive colors and a white background for improved clarity.

3.5.9 Hourly Average Transactions Amounts

The fig 3.7 here visualizes the number of average transactions by creating a line chart representing the average amount of transactions for each hour of the day. in the DataFrame, presumably the `credit_card` by implementing the `px.line()` function from the `plotly.express` library to create a line chart.

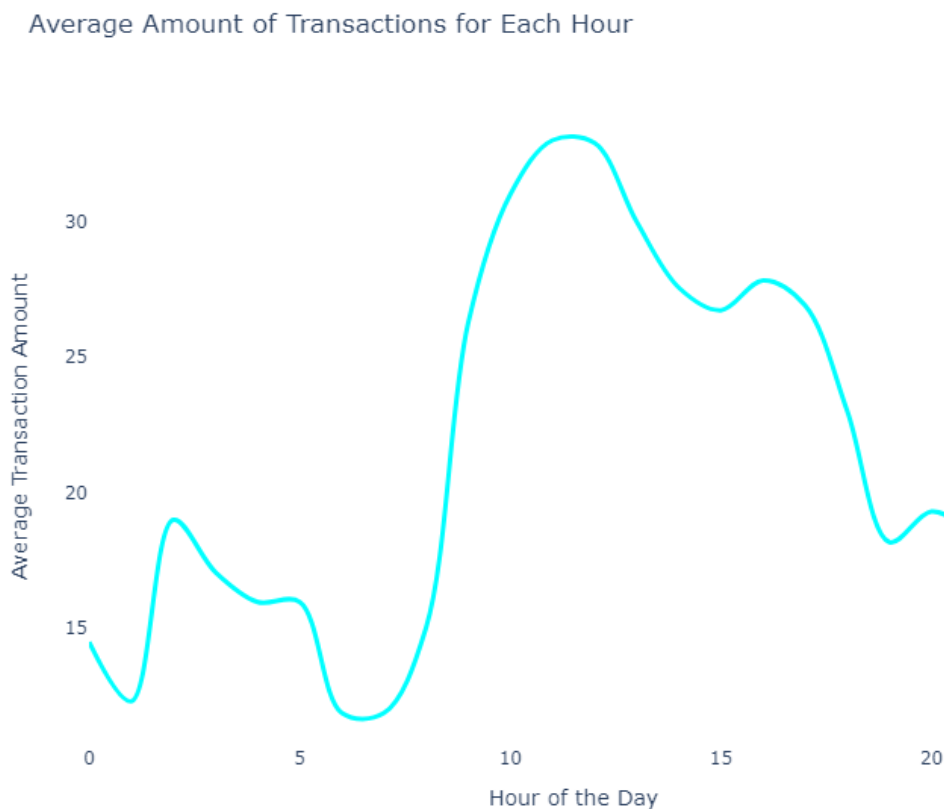


Figure 3.7: Average Transaction Amounts Across Hours

1. Overview:

- Groups the credit card data by the 'Hour' column and calculates the median transaction amount for each hour.
- Utilizes Plotly Express (`px.line`) to create a line chart.

2. Purpose:

- The code aims to visually convey the average transaction amounts throughout the day, providing insights into potential patterns or variations in transaction values.
- By calculating the median transaction amount for each hour and presenting it in a smooth line chart, the code aims to provide insights into potential patterns or variations in transaction values over different hours.

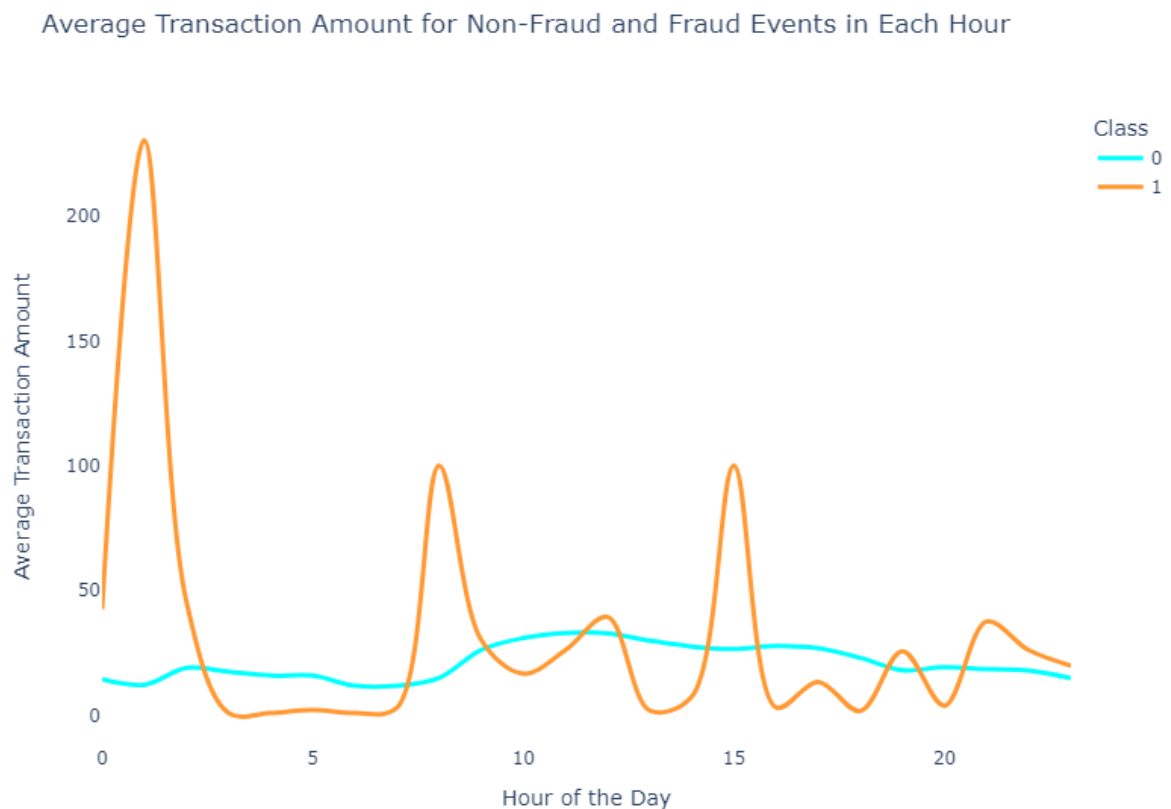


Figure 3.8: Average Fraud and Non-Fraud Transactional Amounts Across Hours

1. Overview:

- Groups the credit card data by both 'Hour' and 'Class' columns, calculating the median transaction amount for each combination.
- Lines are differentiated by color for non-fraudulent (Class 0) and fraudulent (Class 1) transactions.

2. Purpose:

- The code aims to visually compare and analyze the average transaction amounts for non-fraudulent and fraudulent events across different hours of the day.
- The visual representation enhances the interpretability of the data, aiding in the exploration and communication of trends in average transaction amounts over the course of the day.

3.5.10 Histograms of Numerical Features

The fig 3.9 visualizes the numerical features in the dataframe by utilising the `hist()` function.



Figure 3.9: Histogram Visualization of Numerical Features in Credit Card Data

1. Overview:

- The `hist` method is applied directly to the DataFrame (`credit_card`).

- This function generates histograms for each numerical feature in the dataset.

2. Purpose:

- The purpose of this code is to provide a quick visual overview of the distribution of numerical features in the `credit_card` dataset.
- The larger figure size enhances visibility and detail in the visual representation.

3. Insights from histograms:-

- **Transaction Amounts:** The histogram for transaction amounts can reveal the distribution of spending behavior among credit card users. It may show whether most transactions are small, indicating routine purchases, or if there are occasional large transactions, which could indicate significant purchases or outliers.
- **Account Balances:** Histograms for account balances can illustrate the distribution of funds across credit card accounts. They can indicate whether most accounts have low balances, suggesting regular usage and payments, or if there are accounts with consistently high balances, which may indicate potential credit risk or high spending habits.
- **Credit Limits:** Histograms for credit limits can provide insights into the range of credit available to cardholders. They can show whether most cardholders have relatively low or high credit limits, as well as any outliers with exceptionally high or low limits.
- **Transaction Frequencies:** Histograms for transaction frequencies can display the distribution of how often cardholders use their credit cards. They can reveal whether most cardholders make transactions frequently or infrequently, helping to understand overall card usage patterns.
- **Payment Amounts:** Histograms for payment amounts can show how much cardholders typically pay towards their credit card balances. They can indicate whether most payments are small or large, as well as any patterns in payment behavior over time.

By visualizing these histograms, analysts can gain a deeper understanding of the underlying distribution of numerical features in the credit card dataset. This understanding can help identify trends, anomalies, or patterns that may inform business decisions, risk management strategies, or fraud detection algorithms.

3.5.11 Correlation Heatmap

The heatmap used for visualization of the correlation matrix is shown within the fig: 3.10 . Which accounts for the numerical features in the `credit_card` dataset. The pairwise correlations between variables, managing valuable insights into potential relationships are represented with the help of this correlation matrix. The heatmap accounting to a figure size of 12 by 9, provides a clear overview of the strength and direction of correlations, aiding in the exploration of inter-feature connections within the dataset.

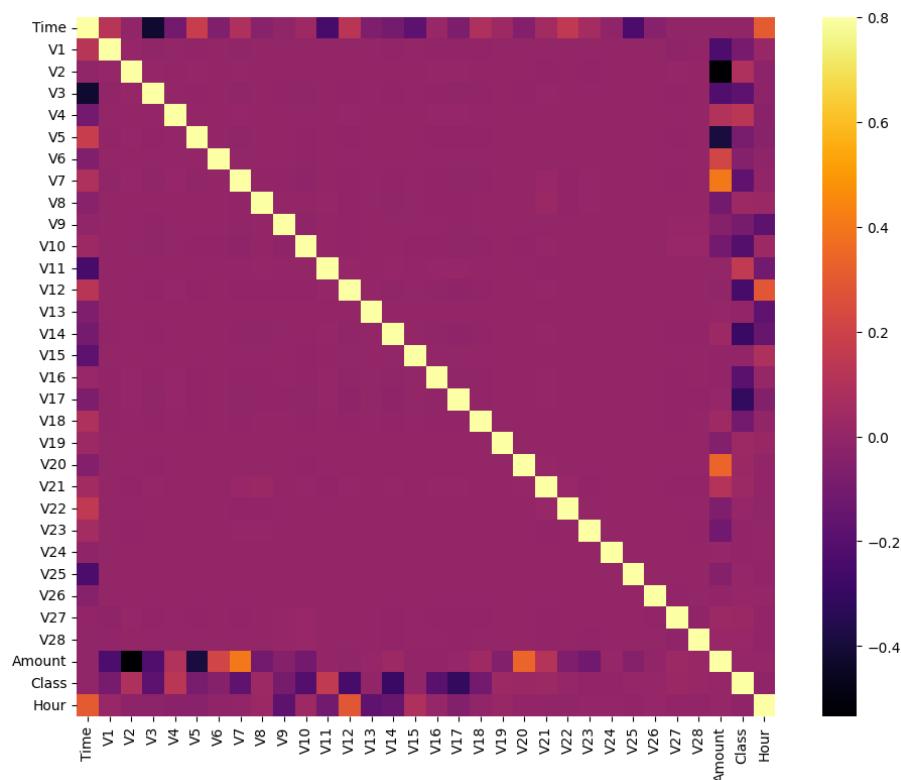


Figure 3.10: Correlation Heatmap

1. Overview:

- Computes the correlation matrix (`corr_matrix`) for numerical features in the `credit_card` dataset.
- Utilizes Seaborn's heatmap function to visually represent the correlation matrix.
- Adjusts the figure size to 12 by 9 for optimal visualization.

2. Purpose:

- The strength and the numerical features and it's direction with respect to it's relation can be easily identified with the help of this heatmap.
- The underlying patterns , relations and correlations, along with the interconnections, can be easily provided in the form a visual representation with the help of this heatmap.
- The performance of the model as well as the feature selection can be identified with the support from the underlying correlated features.
- It vastly helps in EDA(exploratory data analysis) by indulging in a detailed description of the dependencies and the whereabouts of the valuable insights.
- Adjusting the figure size enhances visibility, making it easier to interpret and extract meaningful information from the correlation matrix.

3. Insights from Correlation Heatmap:

- **Strong Correlations:**High positive correlations between certain features may indicate potential relationships or dependencies. For example, strong positive correlations between transaction amounts and account balances could suggest that customers with higher balances tend to make larger transactions, which might be relevant for fraud detection algorithms.
- **Negative Correlations:**Negative correlations between features may also be informative. For instance, a negative correlation between transaction frequency and account age could imply that newer accounts tend to have higher transaction frequencies, which could be a factor in identifying suspicious activity.
- **Correlation with Target Variable:**It's important to examine the correlations between features and the target variable (fraudulent transactions) as well. Features with high positive or negative correlations with the target variable may be strong indicators of fraudulent activity. For example, if there is a strong positive correlation between transaction amount and fraudulent transactions, it could suggest that larger transactions are more likely to be fraudulent.
- **Multicollinearity:**The heatmap can also help identify multicollinearity, which occurs when two or more features are highly correlated with each other. Multicollinearity can affect the performance of predictive models by making the estimates of the coefficients less stable. Identifying and addressing multicollinearity is important for building accurate fraud detection models.

Overall, the correlation heatmap serves as a valuable tool for understanding the relationships between different features in the credit card fraud detection dataset and can provide insights that inform the development of effective fraud detection algorithms.

3.5.12 Pair Plot Analysis

The fig 3.11 visualizes the pai-plot for a select subset of features from the `credit_card` dataframe for a sample of 1000 data points.

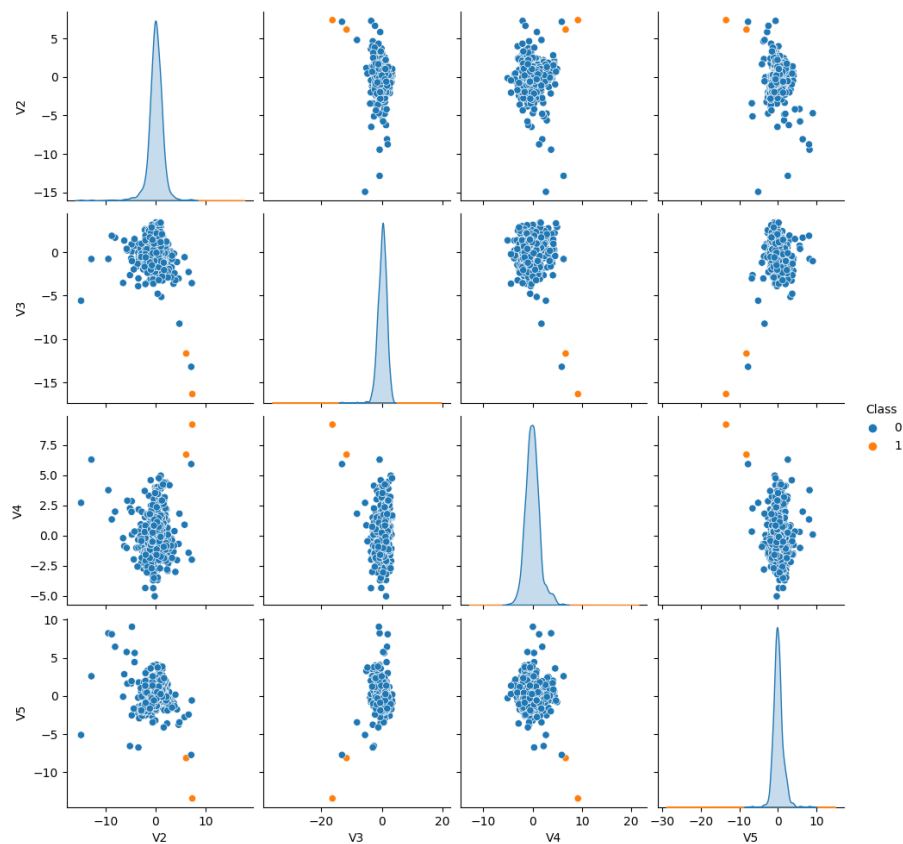


Figure 3.11: Pair-Plot

1. Overview:

- Specifies a subset of features ('V2', 'V3', 'V4', 'V5', 'Class') to be included in the pair plot.
- Randomly samples 1000 data points from the `credit_card` dataset.
- Utilizes Seaborn's `pairplot` function to create a pair plot.

2. Purpose:

- The pair plot visually explores interactions between selected features, providing scatterplots for pairs of variables in the subset.
- Focusing on a subset of features allows for a more detailed examination of specific variables and their relationships.
- The pair plot provides insights into how the selected features contribute to the separation of classes, aiding in exploratory data analysis.

3. Insights from Pair-Plot Analysis:

- **Feature Distribution:** The diagonal plots (kernel density estimation plots) show the distribution of each individual feature. By examining these plots, we can observe the range and shape of the distributions for each feature. This can help identify any outliers or unusual patterns in the data.
- **Feature Relationships:** The scatter plots in the off-diagonal cells show the relationships between pairs of features. By examining these plots, we can identify any linear or nonlinear relationships between features. For example, we might observe clusters or patterns in the data that indicate correlations or dependencies between certain features.
- **Class Distribution:** By coloring the data points based on the 'Class' variable (fraudulent or non-fraudulent transactions), the pair plot allows us to visualize how different combinations of features relate to fraudulent and non-fraudulent transactions. This can help identify any distinct clusters or patterns associated with fraudulent activity.
- **Separation of Classes:** The pair plot may reveal whether fraudulent transactions form distinct clusters or exhibit different distributions compared to non-fraudulent transactions. This can help identify features or combinations of features that are particularly informative for distinguishing between fraudulent and non-fraudulent transactions.
- **Outliers and Anomalies:** By visually inspecting the scatter plots, we can identify any outliers or anomalies in the data that may be indicative of fraudulent activity. Outliers may appear as data points that deviate significantly from the main cluster or distribution of data points.

Overall, the pair plot provides a comprehensive visual overview of the relationships between select features in the credit card dataset and their associations with fraudulent transactions. It can help identify patterns, clusters, and anomalies in the data, providing valuable insights for fraud detection and analysis.

3.5.13 Insights from EDA

1. The dataset is highly imbalanced, as the class for non-fraud transactions constitutes 99.8% of the data
2. It can be clearly seen that the average time for fraudulent transactions is less than fraudulent ones.
3. Following this, average transaction amount is also less in fraudulent transactions compared to non-fraudulent ones.
4. The least number of transactions done are between 3am to 4am
5. The highest number of non-fraud transactions are done around 10pm
6. The highest number of fraud transactions are done at 1am and 11pm
7. Irrespective of fraud and non-fraud class, the minimum average transaction amount is 9.99K, around 5am to 7am. However, the highest average transaction amount is 35 around 10am
8. In case of fraudulent transactions, it can be clearly seen that the highest average amount of transaction is done around 11am with a value of 43. Furthermore, the average transaction amount fluctuates a lot in different hours of the day
9. In case of non-fraud transactions, the average transaction amount doesn't seem to have any noticeable fluctuations throughout the day.

3.6 Data Splitting

Data splitting is a fundamental practice in machine learning to assess the performance of models on unseen data. The process involves dividing the available dataset into two subsets: one for training the model and another for testing its performance. The primary goal is to ensure that the model generalizes well to new, unseen data. Here for this project the split is in the ratio of

80:20 , which means 80% of the data would be used to for the training set and the rest 20% will be used for the validation/testing set.

PROPOSED MODEL

4.1 Description

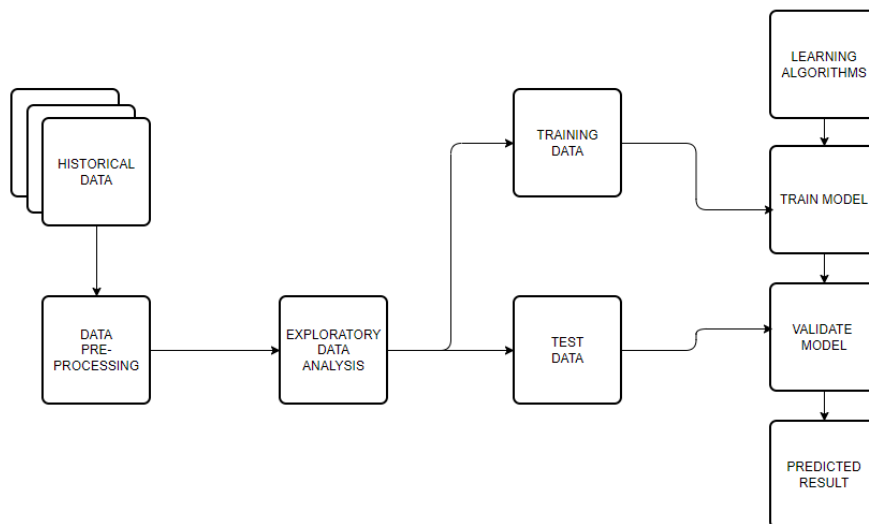


Figure 4.1: System Flowchart

In today's world of technological advancement, the transaction of amounts through credit card has increased by leaps and bounds. In order to keep in check whether the actions are perfect or not we also need a program or a model which can deal with every other obstacle. For such

purpose a high-precision machine learning model is highly required. In our project we have dealt with a huge dataset of credit card transactions to build our model. Building a model as such has more issues in itself which needs to be checked and should be countered properly. The collected data must be preprocessed to ensure that it is clean, consistent, and in a suitable format for analysis. The preprocessing steps include removing null values, dropping duplicated values and make the data more consistent and free of redundancy. We implemented many machine learning features to deal with it, like correlation matrix to understand variability between different variables of the data. The dataset has many features like amount, time, class etc. To build the model the dataset is divided into two sets training and testing sets after preprocessing. The ratio of partitioning is 8:2 i.e 80% for training set and rest 20% for testing set. This splitting technique is used for measuring the level of performance given out by the built machine learning models by implementing different algorithms. We have implemented multiple algorithm to give us the idea of which model would perform the best along certain different conditions. We used six algorithms to build the models namely - Random Forest Classifier Algorithm, Support Vector Machine, Logistic regression, Neural Network, K-Nearest Neighbour & Decision Tree also. The results of predictions are then thoroughly analyzed. This said model is shown in the fig 4.1

4.2 Algorithms

4.2.1 Logistic Regression

Logistic regression is a kind of classification model, which learn and predict the parameters in the given dataset using regression. In logistic regression, the input features are utilised to compute an input weighted sum, which is then passed through a logistic function to determine the likelihood of the binary outcome. Any real-valued input is translated by the logistic function into a value between 0 and 1, using an S-shaped curve. Logistic regression model requires class variable that should be binary classified. Likewise, in this dataset the "target" column has the two type of binary numbers, "0" for the transaction which has less chances of being non-fraudulent, and "1" for the transactions which has more chance of being fraudulent. The independent variables, on the other hand, can be polynomial, nominal, or binary classified.. The logistic regression equation can be written as:

$$P(y = 1|x) = \frac{1}{1 + e^{-z}} \quad (1)$$

The 'Logistic Regression' class from sklearn library is used for implementing logistic regression and it provides several options for regularization, which can help to prevent overfitting and improve the generalization performance of the model.

4.2.2 Support Vector Machine

SVMs are methods, is a basic yet effective Supervised ML method used to generate both regression and classification models. The SVM method works effectively with both linearly and non-linearly separable datasets. In Python, we can use libraries like sklearn. For classification, Sklearn provides functions like SVC, NuSVC & LinearSVC. Here in our project we have used SVC library. Due to their great capacity to generalise to new unseen data objects, lack of local minima, flexible non-linear decision boundary, and dependency on a small number of hyper-parameters, SVM models are commonly utilised in classification tasks. The SVM algorithm's purpose is to find the optimal line or decision boundary for categorising n-dimensional space so that we may simply place fresh data points in the proper category in the future. A hyperplane is the optimal choice of boundary. SVM selects the extreme points/vectors that aid in the creation of the hyperplane. These extreme examples are referred to as support vectors, and the method is known as the Support Vector Machine.

4.2.3 Random Forest

Leo Breiman and Adele Cutler created the Random Forest machine learning algorithm, which combines the output of many decision trees to produce a single conclusion. The few of the main reason to use random forest is that it can easily deal with the usability and verstality that it offer, it really great, also the value we can achieve from the ability to handle the classification and few other challeneges regarding regression of the model is really good. A popular approach for regression and classification is Random Forest. In order to detect the credit card fraud it is crucial to be able to distinguish between the classes and categorise observations for it's varied applications , such as forecasting a user's propensity to purchase a product or to fail on a loan. In simple words, Random Forest is a classifier upon which many decision trees are built on different dataset subsets and averages them to improve the dataset's projected accuracy. We have implemented the Random Forest algorithm in our model by utilising the RandomForestClassifier class from sklearn library. The random forescast algorithm is an extension of the bagging method as it utilizes both bagging and feature randomness to create an uncorrelated forest of decision trees. For this algorithm it's easy to determine the feature importance and are used to measure model accuracy.

4.2.4 Neural Networks :

The neural network architecture defined consists of three main types of layers: the input layer, two hidden layers, and the output layer. Each layer contains a certain number of neurons, which are interconnected and responsible for processing information. The neural network architecture consists of three main types of layers: the input layer, two hidden layers, and the output layer. The input layer serves as the entry point for the input data and has a size determined by the number of features present in the dataset. The two hidden layers are densely interconnected, with 64 neurons in the first hidden layer and 32 neurons in the second hidden layer. These hidden layers are responsible for extracting and learning meaningful representations from the input data. Finally, the output layer consists of a single neuron and produces the final classification prediction (fraudulent or legitimate transaction). Each layer consist of neurons (illustrated as circles in Figure) and in turn each neuron has a weight value w and bias value b . The output vector of each layer is computed by:

$$s(i) = W_i x^{(i-1)} + b_i \quad (2)$$

$$x_i = f(s^i) \quad (3)$$

where $f : \mathbb{R} \rightarrow \mathbb{R}$ is an activation function and x_i are the input values. Conventionally, the hyper parameters w and b are initialized with random values, and during the training process, the values are updated. The output of the network is calculated in a process called forward pass where the input vector x is fed into the network which calculates the output of the layers starting from the first layer up to the last output layer using the Equations. The values of the weights and biases are updated during the training of the model, which is an iterative process. Samples of training data with known input and output values are used for training the algorithm. The output for each training sample is calculated by performing the forward pass. In binary classification, the output layer can be designed with two neurons, where each neuron correspond to one class. The level of activation (high or low output value) of each neuron indicate which class the input belongs to. During the training process, the loss function is minimized by using gradient descent, or other optimization algorithms, to adjust the weights and biases of the network. There are different loss functions that can be used for different problems. Generally, the binary cross-entropy loss function is used in binary classification problems which is defined as:

$$L = -\frac{1}{n} \sum_{j=1}^n (y_j \cdot \log(\hat{y}_j) + (1 - y_j) \cdot \log(1 - \hat{y}_j)) \quad (4)$$

where n is the output size, $\hat{y}_j$ is the output from the j :th neuron with its corresponding target value $\hat{y}_j$. In a process called back-propagation, the loss is minimized by adjusting the weights and biases of the network by first calculating the derivative of the loss function with respect to the biases and weights $\frac{\partial L}{\partial W_i}$ & $\frac{\partial L}{\partial b_i}$. These derivatives are solved by applying the chain rule. After calculating the derivatives, the update in the weights and biases are calculated by:

$$\Delta W_i = -\eta \frac{\partial L}{\partial W_i} \quad (5)$$

$$W_{i+1} = W_i + \Delta W_i \quad (6)$$

where η is the learning rate.

The proposed neural model is illustrated with the fig 4.2.

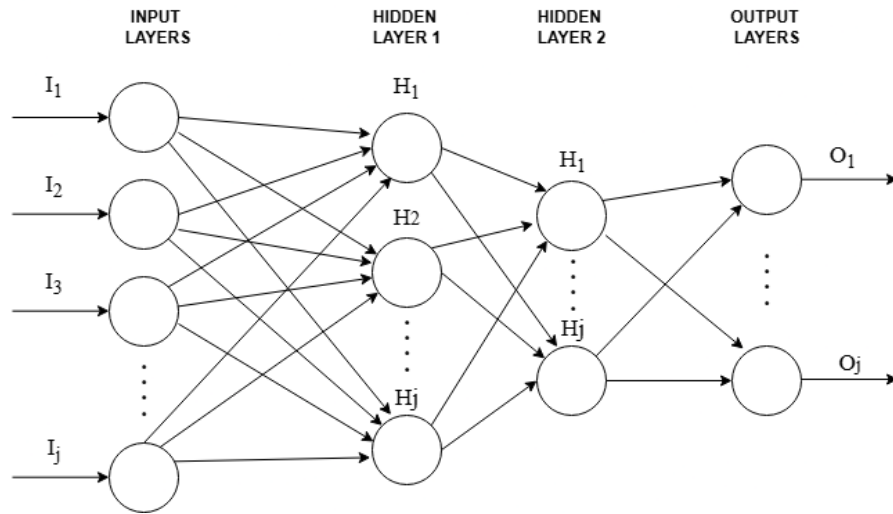


Figure 4.2: Neural Network Illustration

4.2.5 K-Nearest Neighbour Algorithm(KNN) :

The K-nearest neighbors (KNN) calculation is a flexible and natural strategy established in the rule that things in a specific category or course will likely be found close to other things in that same category or lesson. At its center, KNN is about leveraging the control of nearness, utilizing the thought that comparative information focuses will cluster together. KNN holds a noteworthy position in the field of machine learning and information science, fundamentally for its straightforwardness and viability. Whereas it's overwhelmingly known for its applications

in directed learning, especially classification, it has been utilised in semi-supervised scenarios where information might be mostly labeled. One of the standout viewpoints of KNN is its capacity to adjust to non-linear choice boundaries, making it flexible when managing complex datasets. Choosing 'k': Start by choosing the number of neighbors,'k'. The choice of 'k' is vital. A littler 'k' can be boisterous and vulnerable to exceptions,though a huge 'k' can smooth the choice boundaries excessively.

4.2.6 Decision Tree :

Decision tree procedure is factual information mining procedure in which free and subordinate properties are coherently communicated in a structure in the frame of a tree.The categorization rules determined from the choice tree are if at that point expressions, and to produce each run the show, all tests must pass. Choosing trees as a rule part a complex issue into numerous basic ones, and utilize emphasis to unravel sub- issues. The tree is a prescient choice back instrument that makes mappings of conceivable results from distinctive perceptions. There are various noticeable classifiers for producing course models from decision trees.

4.3 Experimental Setup

4.3.1 Import Libraries

1. **NumPy:** NumPy is a numerical computing library for Python which provides a support of multiple features to deal with.A multidimensional array object, different derived objects (like masked arrays and matrices), and a variety of routines for quick array operations—like sorting, selecting, I/O, discrete Fourier transforms, basic linear algebra, basic statistical operations, random simulation, and much more—are all provided by this Python library.
2. **Matplotlib:** It is a complete Python visualisation toolkit for static, animated, and interactive graphics.It is a cross-platform tool with several applications, including graphical charting (bar charts, scatter plots, and histograms) and data plotting. Matplotlib creates interactive settings and figures of publishing quality for a range of hardcopy formats and platforms. Web application servers, Python scripts, Python/IPython shells, and other graphical user interface toolkits may all utilise Matplotlib.
3. **Pandas:** This is an open-source, robust, and flexible Python module used for data analysis and manipulation.Pandas need less code to access many of the functions provided

by NumPy and matplotlib. For example, you may plot a chart in a few lines using pandas'.plot() function, which combines several matplotlib functions into a single method. It offers high-level, user-friendly capabilities, mostly in the form of DataFrames, which are tabular data. We can import, clean, convert, and analyse data quickly and effectively with Pandas. It provides features for statistical analysis, grouping, filtering, selecting, and indexing of data.

4. **Scikit learn:** scikit-learn is a Python module for machine learning built on top of SciPy and is distributed under the 3-Clause BSD license. Scikit-learn provides a range of supervised and unsupervised learning algorithms via a consistent interface in Python. The goal of the library is to reach a support and robustness level necessary for use in production systems. This entails giving serious consideration to issues like performance, documentation, collaboration, code quality, and ease of use.
5. **Plotly:** Plotly Express, or PX, also known as the plotly.express module (typically imported as px), which provides functions capable of creating whole figures at once. The suggested place to start when making the majority of popular figures is Plotly Express, which is an integrated component of the plotly library. Plotly express is a high-level data visualization package that allows you to create interactive plots with very little code.
6. **Imbalanced Learn :** This is a Python library designed to address the challenges posed by imbalanced datasets in machine learning. Imbalanced datasets occur when the distribution of classes in the target variable is skewed, leading to issues in training models that may be biased towards the majority class. Imbalanced-learn provides various techniques for handling imbalanced data, including oversampling the minority class, undersampling the majority class, and generating synthetic samples using methods like SMOTE (Synthetic Minority Over-sampling Technique).
7. **Seaborn :** Seaborn (styled as “seaborn”) is an open-source Python library used for visualizing the explorative statistical plots of data. It is a basic library used to extract various information that can provide an understanding of a dataset. It is built on top of matplotlib and is used for informative and easily understandable statistical graphics.

4.3.2 Model Evaluation

1. **Logistic Regression :** One type of supervised learning method is the logistic regression algorithm. This model is used for a binary classification problem in our project. Here, the

procedure starts with importing the scikit-learn library's logistic regression class. The `fit` technique is then used, using the inputs as `X_train` for training features and `y_train` for matching labels, because the model is now prepared for training. Following the training phase, the model is given the responsibility of generating predictions using the testing data, which is a different collection of data. Here, the testing features (`x_test`) are subjected to the `predict` algorithm, which yields a set of predicted labels that are saved in the `lg_pred` variable. This process's capacity to forecast fresh, unknown data based on the patterns it has learnt from the training set is what gives it practical relevance. Metrics like accuracy, precision, recall, and others may be assessed by comparing these predictions (`lg_pred`) to the true labels of the testing set (`y_test`). This allows for an evaluation of the model's performance. What gives this approach practical value is its ability to forecast new, unknown data based on the patterns it has learned from the training set. By comparing the actual labels of the testing set (`y_test`) with these predictions (`lg_pred`), metrics such as accuracy, precision, recall, and others may be evaluated. This makes it possible to assess the model's effectiveness.

2. **Support Vector Machine :** Support vector machines (SVMs) are reliable techniques for both linear and non-linear classification when a linear kernel is selected, as in this case. The Support Vector Machine (SVM) is implemented using the scikit-learn Support Vector Classification (SVC) class, with the regularisation parameter 'C' set at 0.1. As before, the `fit` technique is used to train the model using the training features `X_train` and matching labels `y_train`. At this point, the programme finds the hyperplane by optimising its parameters. Following training, the model is applied to the testing features `X_test` using the 'predict' approach. The final predictions are stored in the `svc_pred` variable. By comparing the predicted labels with the actual labels `y_test`, one may evaluate the model's performance using many metrics, including accuracy, precision, recall, and others.
3. **Random Forest :** The Random Forest algorithm is a versatile ensemble learning method that combines the predictions of multiple decision trees to enhance overall predictive accuracy and generalization. The `RandomForestClassifier` is implemented at various steps first being the initialization where specific settings are altered : it will consist of 100 decision trees (`n_estimators=100`), and a random seed (`random_state=42`) is set to ensure reproducibility of results. Next, training of the classifier is started while using a resampled set of training data. Resampling is a technique commonly used to handle imbalances in class distribution within the training dataset. Here, the features of the resampled training set is contained within `X_train_res`, while `y_train_res` holds

the corresponding labels. During training, the algorithm learns various patterns and relationships that are definite and intricate within the data, benefiting from the collective insights of multiple decision trees. After training, the classifier is applied to new, unseen data represented by `X_test`. The `predict` method is employed to generate predicted labels, stored in the variable `y_pred`. The true labels of the test set are then compared with the predicted labels, allowing for the evaluation of the classifier's performance and its ability to generalize knowledge from the training data to previously unseen instances. This process is crucial in assessing the model's effectiveness in making accurate predictions on new data.

4. **Neural Network :** Incorporating neural networks into the realm of credit card fraud detection represents a significant advancement in bolstering financial security. These sophisticated algorithms, drawing inspiration from the intricate workings of the human brain, demonstrate remarkable capabilities in analyzing extensive volumes of transactional data, identifying subtle irregularities indicative of fraudulent activities. To ensure a robust evaluation of the model's performance on unseen data, the dataset is divided into training and testing sets using the `'train_test_split'` function, with 80% allocated for training (`'X_train, y_train'`) and 20% for testing (`'X_test, y_test'`).

The neural network architecture comprises three fully connected (Dense) layers: The initial layer, with 64 units and a Rectified Linear Unit (ReLU) activation function, expects input data shaped as per the number of features in the training set (`X_train.shape[1]`). The subsequent layer, with 32 units and ReLU activation, follows suit. The final layer, encompassing a single unit with a sigmoid activation function, facilitates binary classification tasks by producing output probabilities. Through the `fit` method, the model is trained on the designated training data (`'X_train, y_train'`), endeavoring to minimize the specified loss function.

Post-training, predictions are generated on the test set (`'X_test'`) using the `predict` method. This process calculates the probability of each instance belonging to each class. Subsequently, the `'np.argmax'` function is employed to determine the final predicted class labels, selecting the class with the highest predicted probability for each instance. By adopting this approach, the model effectively assigns the most probable class label to each data point within the test set.

5. **K-Nearest Neighbour :** When implementing K-Nearest Neighbors (KNN) for credit card fraud detection, the process involves utilizing the algorithm's proximity-based approach

to classify transactions as either legitimate or fraudulent. Feature scaling is performed using **StandardScaler** to standardize the features. This ensures that all features contribute equally to the distance calculations in the KNN algorithm. A KNN classifier (`knn`) is initialized with `n_neighbors=5`. The classifier is trained on the scaled training data (`X_train_scaled`, `y_train`) using the `fit` method. To assess the performance of the KNN model for credit card fraud detection, various evaluation metrics are utilized. These include the confusion matrix, which provides insights into the true positive, true negative, false positive, and false negative predictions. By leveraging the algorithm's proximity-based classification methodology, financial institutions can improve their ability to identify fraudulent transactions in real-time, thereby minimizing losses and safeguarding consumer assets.

6. **Decision Tree :** To build a decision tree for credit card fraud detection, recursively split the dataset based on attributes that give the highest information gain or purity. At each node, the algorithm chooses the attribute that best distinguishes fraudulent from valid transactions, resulting in a hierarchical tree structure. By iteratively separating the data based on decision criteria, the algorithm builds branches that steer the classification process, eventually giving class labels to individual transactions at the leaf nodes. A decision tree classifier (`clf`) is initialized with `random_state=42` for reproducibility. The classifier is trained on the scaled training data (`X_train_scaled`, `y_train`) using the `fit` method. Predictions are made on the scaled test set (`X_test_scaled`) using the `predict` method. By leveraging decision rules derived from the dataset, decision trees can effectively classify transactions as either fraudulent or legitimate.

RESULTS AND ANALYSIS

5.1 Classification Metrics

5.1.1 Confusion Matrix

One approach for assessing a classification model's performance is a confusion matrix. It contrasts the machine learning model's projected values with the actual target values. In our use case, we are attempting to forecast the quantity of transactions that are fraudulent and those that are not. The Test dataset, which makes up 20% of the entire dataset, is what we will use to test our model. The suggested Models will attempt to forecast these, and the confusion matrix will give the full classification report. We can compute the following crucial classification metrics with this tool: recall, accuracy, precision, and F1 score.

Accuracy: It is the ratio of the number of correct predictions to the total number of input samples. It works well only if there are equal number of samples belonging to each class.

$$\text{Accuracy} = \frac{\text{Number of Correct Predictions}}{\text{Total Number of Predictions}}$$

Precision(Weighted): It is defined as the proportion of accurately anticipated positive observations to all positive predictions. It makes sense that the classifier's precision is its capacity to reject labelling a negative sample as positive. The weighted approach determines the average of the metrics for each label, weighted by the total number of true cases for each label.

$$\text{Precision (Weighted)} = \frac{1}{\sum_{i=1}^n w_i} \sum_{i=1}^n w_i \cdot \frac{TP_i}{TP_i + FP_i} \quad (7)$$

where n is the number of classes, w_i is the number of true occurrences for each label, TP_i is the number of true positives for each class, and FP_i is the number of false positives for each class.

Recall(Weighted): It is the ratio of correctly anticipated positive observations to all of the class's observations. The classifier's innate ability to find every positive sample is known as recall. The weighted method calculates metrics for every label and divides the total number of true cases by the total number of labels.

$$\text{Recall (Weighted)} = \frac{1}{\sum_{i=1}^n w_i} \sum_{i=1}^n w_i \cdot \frac{TP_i}{TP_i + FN_i} \quad (8)$$

where w_i is the number of true instances for each label, n is the number of classes, TP_i is the number of true positives for each class, and FN_i is the number of negatives for each class.

F1 Score(Weighted): The F1 score may be viewed as a weighted average of accuracy and recall, with a greatest value of 1 and a worst value of 0. The weighted technique computes metrics for each label and gets the average of those metrics weighted by the number of true cases for each label.

$$\text{F1 Score (Weighted)} = \frac{1}{\sum_{i=1}^n w_i} \sum_{i=1}^n w_i \cdot \frac{2 \cdot (\text{precision}_i \cdot \text{recall}_i)}{\text{precision}_i + \text{recall}_i} \quad (9)$$

where w_i is the number of true instances for each label, n is the number of classes, Precision_i is the precision for each class and Recall_i is the recall for each class

5.2 Analysis of Results

In this section all the experimental results of different algorithms applied while building the model have been discussed along with all others takeaways from the previous works.

5.2.1 ROC AUC Metrics

The ROC curve and the AUC Curve are two extensively used graphical measures for evaluating the performance of binary classification algorithms. For various threshold settings, it plots the True Positive Rate (Sensitivity) vs the False Positive Rate (1 - Specificity). The area under the ROC curve (AUC) is a single scalar statistic that summarises the model's overall performance across various threshold settings.

The below table records all the ROC AUC Score of different algorithms use

Algorithms	ROC AUC Scores
Logistic Regression	0.77
Support Vector Machine	0.87
Random Forest	0.89
Neural Network	0.99
KNN	0.91
Decision Tree	0.86

Table 1: ROC AUC Metrics

The scores obtained for each algorithm represent the performance of each algorithm on a specific task, likely a classification task, based on the ROC AUC metric. Let's discuss the potential reasons behind the scores and their implications:

- **Support Vector Machine (SVM) - 0.77:** SVM might have scored lower due to its sensitivity to the choice of kernel function and regularization parameters. If the data is not linearly separable or if the kernel function is not chosen appropriately, SVM's performance may degrade.
- **Logistic Regression - 0.87:** Logistic Regression is a simple yet effective algorithm for binary classification tasks. The score of 0.87 suggests that it's performing reasonably well on the given task. However, it might not capture complex relationships in the data as effectively as more sophisticated algorithms like neural networks.

- **Random Forest - 0.89:**Random Forest is an ensemble learning method known for its robustness and ability to capture complex relationships in the data. The score of 0.89 indicates that Random Forest is performing well and likely providing good generalization on the task.
- **Neural Network - 0.99:**Neural networks, especially deep neural networks, are capable of learning highly complex patterns in the data. A score of 0.99 suggests that the neural network has learned the task almost perfectly, achieving near-perfect discrimination between the classes. This high score might indicate that the neural network architecture is well-suited for the task and/or that the model has overfit the training data.
- **K-Nearest Neighbors (KNN) - 0.91:**KNN is a non-parametric algorithm that makes predictions based on the similarity of data points. The score of 0.91 suggests that KNN is performing well on the task, likely because it's capturing local patterns effectively. However, it can be computationally expensive for large datasets and sensitive to the choice of the number of neighbors (k).
- **Decision Tree - 0.86:**Decision Trees are simple and interpretable models that partition the feature space based on if-else conditions. The score of 0.86 indicates that the decision tree is providing reasonable discrimination between classes. However, decision trees can be prone to overfitting, especially if the tree grows too deep.

In summary, the scores obtained by each algorithm reflect their respective strengths and weaknesses in capturing patterns in the data. A higher score suggests better discrimination between classes, while a lower score may indicate that the algorithm struggles to capture the underlying patterns effectively.

5.2.2 Confusion Matrix Analysis

1. **Logistic Regression:** The figure 5.1 shows the values of the Confusion Matrix of all

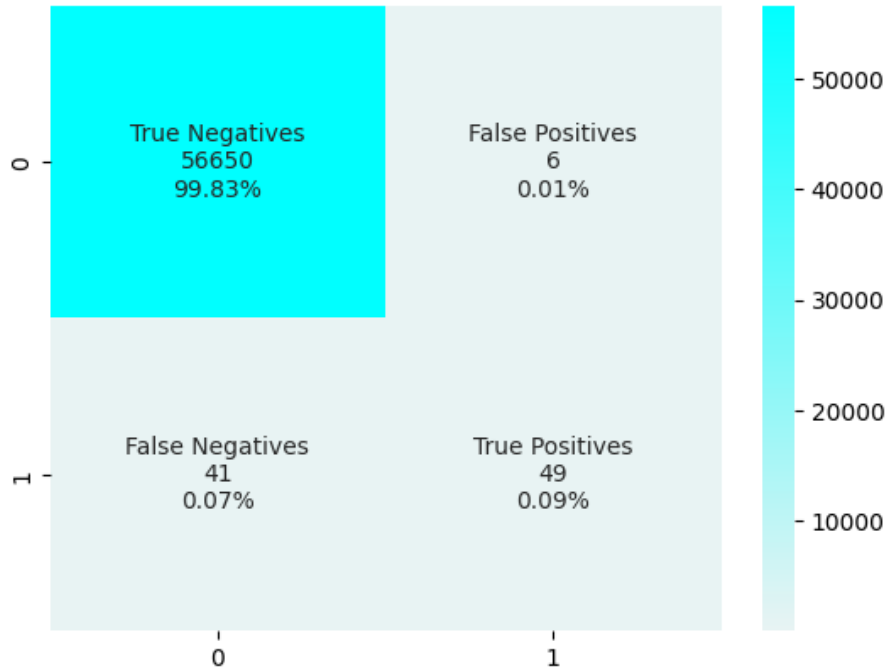


Figure 5.1: Confusion Matrix for Logistic Regressor

possible predicted outcomes for Logistic Regression. The values along the diagonal from top left to bottom right represents the samples which are correctly classified. Rest of the values represent miss classification. The confusion matrix for the logistic regression model reveals its performance:

- **True Negatives(TN):**The logistic regression model accurately identified 99.83% of legitimate transactions as non-fraudulent, demonstrating its effectiveness in recognizing normal activities.
- **False Positive(FP):**With only 0.01% of legitimate transactions misclassified as fraudulent, the model exhibits a low false positive rate, minimizing unnecessary alerts to customers.
- **False Negative(FN):**Missing 0.07% of fraudulent transactions, the model should aim to reduce false negatives to enhance fraud detection and prevent potential financial losses.
- **True Positive(TP):**Successfully detecting 0.09% of fraudulent transactions, the model demonstrates its capability to identify actual instances of fraud.

2. **Support Vector Machine:** The figure 5.2 shows the values of the Confusion Matrix of

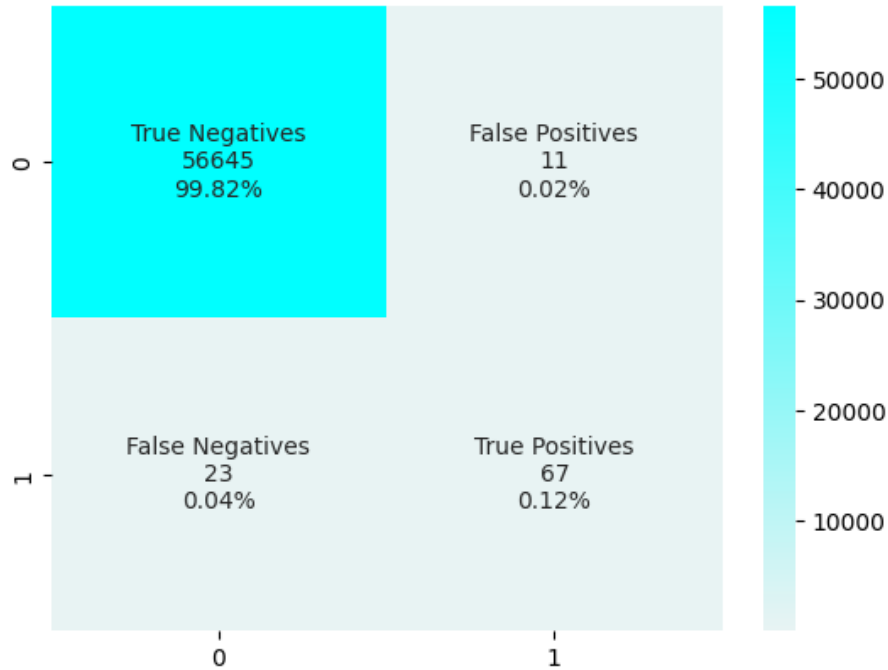


Figure 5.2: Confusion Matrix for SVM

all possible predicted outcomes for SVM. The values along the diagonal from top left to bottom right represents the samples which are correctly classified. Rest of the values represent miss classification.

- **True Negatives(TN):**The model correctly identified legitimate transactions as non-fraudulent with a high accuracy of 99.82%. This indicates the system's effectiveness in distinguishing normal, non-fraudulent activities.
- **False Positive(FP):**The low rate of false positives (0.02%) suggests that the system rarely misclassifies legitimate transactions as fraudulent, reducing unnecessary alerts to customers and operational costs.
- **False Negatives (FN):**Although the rate of false negatives (0.04%) is relatively low, it's crucial to minimize missed fraudulent activities to enhance security and prevent financial losses for both customers and the institution.
- **True Positives (TP):**The model correctly flagged a small percentage (0.12%) of transactions as fraudulent, indicating its ability to detect actual instances of fraud.

3. Random Forest:

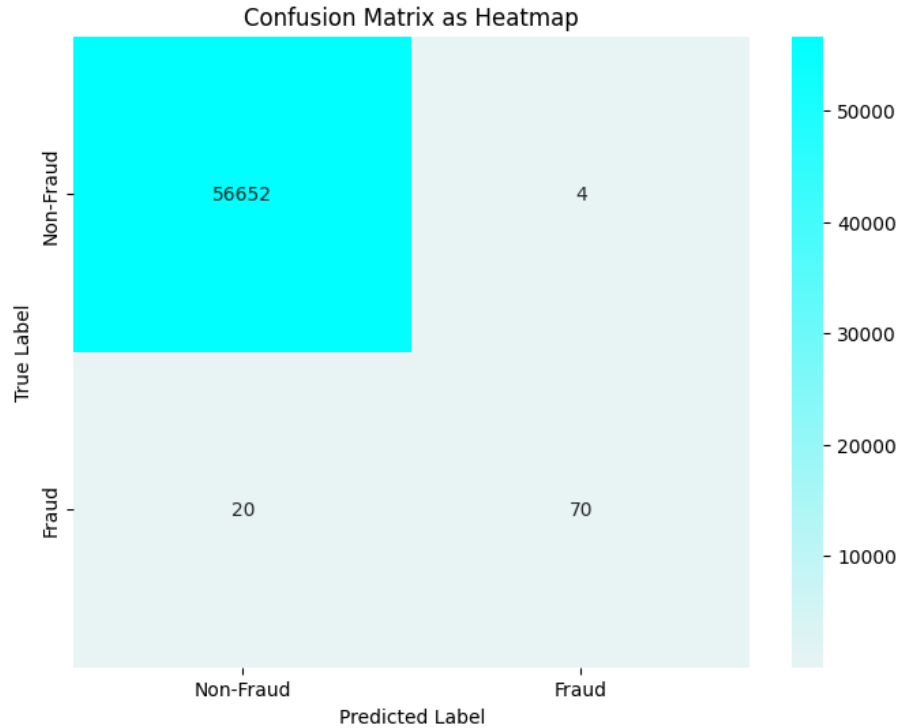


Figure 5.3: Confusion Matrix for Random Forest

The figure 5.3 shows the values of the Confusion Matrix of all possible predicted outcomes for Random Forest. The values along the diagonal from top left to bottom right represents the samples which are correctly classified. Rest of the values represent miss classification.

- **True Negatives(TN):** Identified 99.99% of legitimate transactions correctly as non-fraudulent, indicating strong accuracy in recognizing normal activities.
- **False Positive(FP):** Misclassified only 4 legitimate transactions as fraudulent, demonstrating a low false positive rate and minimizing unnecessary alerts.
- **False Negative(FN):** Missed 20 fraudulent transactions, suggesting a need to reduce false negatives to enhance fraud detection efficiency.
- **True Positive(TP):** Successfully detected 70 fraudulent transactions, showcasing the model's capability to identify instances of fraud.

4. **Neural Network:** The figure 5.4 shows the values of the Confusion Matrix of all possible

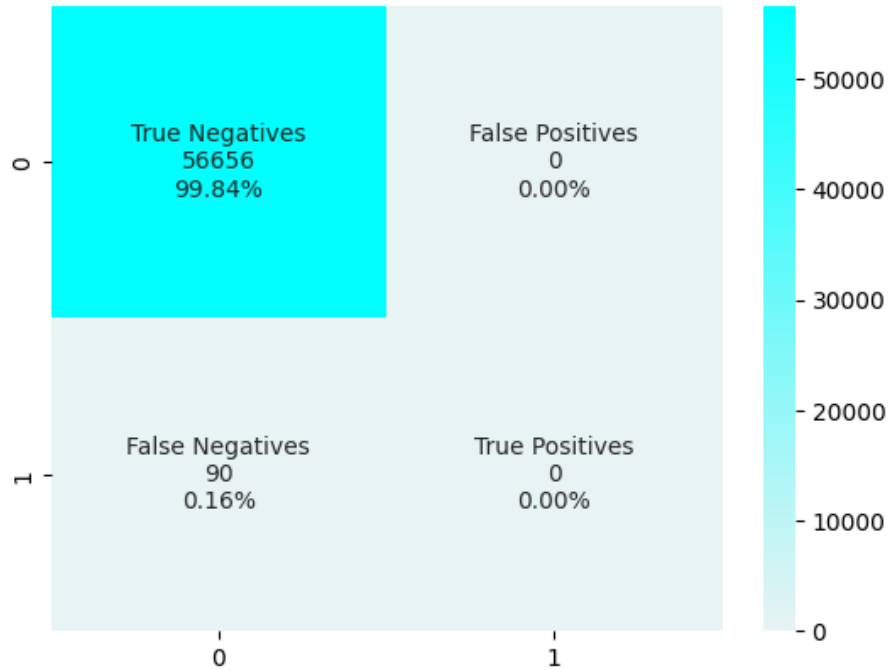


Figure 5.4: Confusion Matrix for Neural Network

predicted outcomes for Neural Network. The values along the diagonal from top left to bottom right represents the samples which are correctly classified. Rest of the values represent miss classification.

- **True Negative(TN):** Identified almost all legitimate transactions correctly as non-fraudulent, indicating high accuracy in recognizing normal activities.
- **False Positive(FP):** No legitimate transactions were incorrectly classified as fraudulent, demonstrating a perfect precision and no unnecessary alerts.
- **False Negative(FN):** Missed 90 fraudulent transactions, suggesting a need to improve sensitivity to detect fraudulent activities more effectively.
- **True Positive(TP):** No fraudulent transactions were correctly identified, indicating a significant limitation in the model's ability to detect instances of fraud.

5. **K-Nearest Neighbour**: The figure 5.5 shows the values of the Confusion Matrix of all

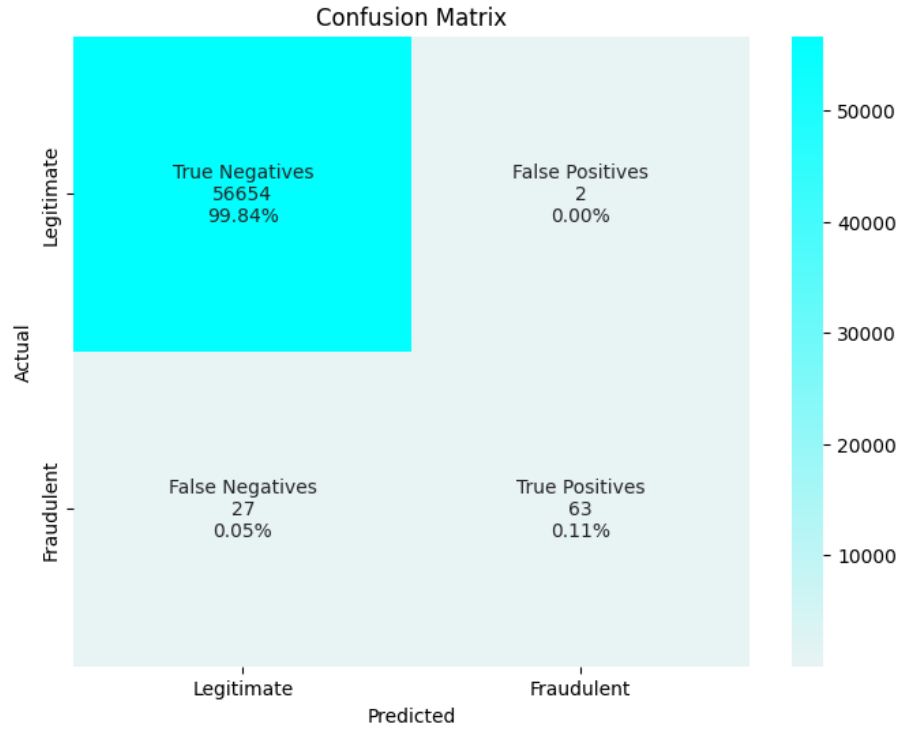


Figure 5.5: Confusion Matrix for KNN

possible predicted outcomes for KNN(K-Nearest Neighbour). The values along the diagonal from top left to bottom right represents the samples which are correctly classified. Rest of the values represent miss classification.

- **True Negative(TN)**:Successfully identified 99.99% of legitimate transactions as non-fraudulent, indicating strong accuracy in recognizing normal activities.
- **False Positive(FP)**:Misclassified only 2 legitimate transactions as fraudulent, demonstrating a very low false positive rate and minimizing unnecessary alerts.
- **False Negative(FN)**: Missed 27 fraudulent transactions, suggesting a need to reduce false negatives to enhance fraud detection efficiency.
- **True Positive(TP)**:Successfully detected 63 fraudulent transactions, indicating the model's capability to identify instances of fraud.

6. Decision Tree:

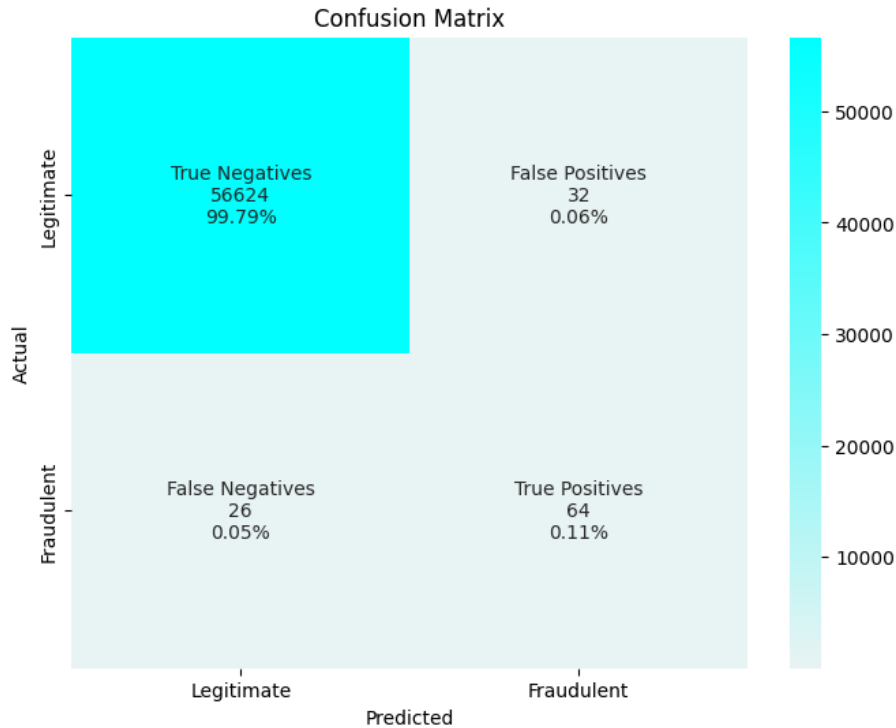


Figure 5.6: Confusion Matrix for Decision Tree

The figure 5.6 shows the values of the Confusion Matrix of all possible predicted outcomes for Decision Tree. The values along the diagonal from top left to bottom right represents the samples which are correctly classified. Rest of the values represent miss classification.

- **True Negative(TN):** Identified 99.94% of legitimate transactions correctly as non-fraudulent, indicating strong accuracy in recognizing normal activities.
- **False Positive(FP):** Misclassified 32 legitimate transactions as fraudulent, implying a higher false positive rate compared to other models, potentially leading to some unnecessary alerts to customers.
- **False Negative(FN):** Missed 26 fraudulent transactions, suggesting room for improvement in detecting instances of fraud to enhance security.
- **True Positive(TP):** Successfully detected 64 fraudulent transactions, demonstrating the model's capability to identify some instances of fraud.

5.2.3 Model Tuning

Model tuning is the process of determining the best hyperparameter values to optimise model performance. Tuning a model helps it function optimally. This procedure entails modifying different parts of the model to attain optimal results. We employed K-fold cross validation in model tuning to produce accurate and fair estimates of model performance across different data subsets. This approach divides the dataset into k subsets (folds) and trains and validates the model k times, each time using a different fold as the validation set.

To summarise, k-fold cross-validation is a methodical way to determining the best hyperparameter configuration for a machine learning model. By employing several validation sets and measuring performance measures, this procedure guarantees that the chosen model is both accurate and resilient across varied subsets of the data, permitting enhanced generalisation to unknown occurrences.

5.2.4 Analysis of Model Tuning

K-Folds Cross Validation The k-fold cross-validation optimizes the performance and ensures robustness of a machine learning model. Cross-validation is a technique used to assess how well a model will generalize to an independent dataset. The "k" in k-fold cross-validation represents the number of folds or subsets that the dataset is split into. K-fold Cross-Validation with $K = 5$, was used for all the models and the validation accuracy across all the folds is given in this chapter along with that the average cross-validated F1-Score is also provided.

1. **Logistic Regression** The following table 2 shows the results which are obtained by performing k-fold cross validation on SVM :-

No. of Folds(K)	F1 Score
1st	0.8569443
2nd	0.8092591
3rd	0.87390768
4th	0.87294681
5th	0.87575785

Table 2: K-Folds for Logistic Regression

The average K-Fold cross validated F1-Score for Logistic Regression is **0.8577631525920651**.

2. **Support Vector Machine** The following table 3 shows the results which are obtained by performing k-fold cross validation on SVM :- The average K-Fold cross validated

No. of Folds(K)	F1 Score
1st	0.91081549
2nd	0.86965341
3rd	0.91757808
4th	0.91536058
5th	0.8985218

Table 3: K-Folds for SVM

F1-Score for SVM is **0.9023858727966685** .

3. **Random Forest** The following table 4 shows the results which are obtained by performing k-fold cross validation on Random Forest :- The average K-Fold cross validated F1-

No. of Folds(K)	F1 Score
1st	0.99985657
2nd	0.9999338
3rd	0.99987864
4th	0.99991174
5th	0.99987864

Table 4: K-Folds for Random Forest

Score for Random Forest is **0.9998918785026693** .

4. **Neural Network** The following table 5 shows the results which are obtained by performing k-fold cross validation on Neural Network :- The average K-Fold cross validated

No. of Folds(K)	F1 Score
1st	0.78251103
2nd	0.49957559
3rd	0.82449228
4th	0.49957559
5th	0.55983858

Table 5: K-Folds for Neural Network

F1-Score for Neural Network is **0.6331986152537065** .

5. K-Nearest Neighbour

No. of Folds(K)	F1 Score
1st	0.99949335
2nd	0.99933915
3rd	0.99960349
4th	0.99955943
5th	0.99940523

Table 6: K-Folds for KNN

The average K-Fold cross validated F1-Score for KNN is **0.9994801304079655** .

6. Decision Tree

No. of Folds(K)	F1 Score
1st	0.82403749
2nd	0.86751576
3rd	0.87799550
4th	0.88488904
5th	0.87084300

Table 7: K-Folds for Decision Tree

The average K-Fold cross validated F1-Score for Decision Tree is **0.8650561567337072**

Thus from the above table 2 to table 7 it is clear that Random Forest is having the highest F1-Score among all i.e **0.9998918785026693** .

5.2.5 Accuracy Result Analysis

The table‘8 below provides all the models’ accuracy implemented till now.

Classifier	Accuracy(%)
Logistic Regression	99.92
SVM	99.94
Random Classifier	99.96
Neural Network	99.84
KNN	99.95
Decision Tree	99.89

Table 8: Accuracy Details

5.2.6 Model Comaprison

The bar graph 5.7 below shows the different accuracy of all the algorithms implemented.

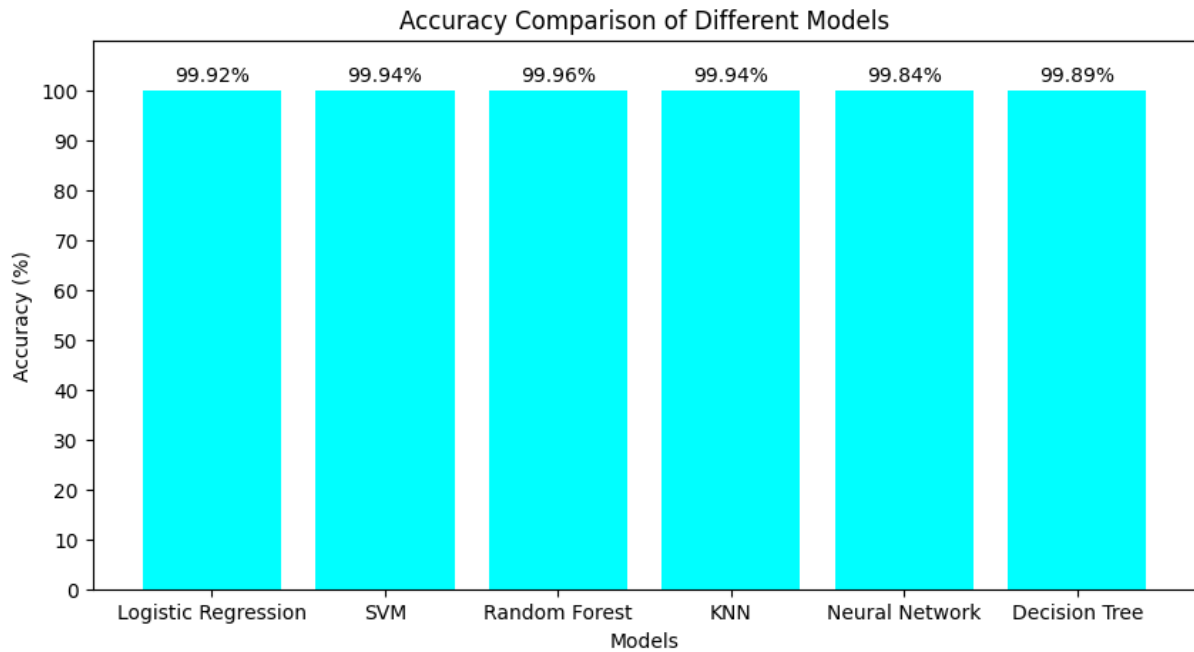


Figure 5.7: Accuracy Barplot for different models

From table 8 and figure 5.7, it can be concluded that Random Forest Classifier is the best model as it is having the highest accuracy, So we applied this model in test data for prediction.

It will give 99.96% accuracy in the testing data set.

5.2.7 Statistical Significance Test

The goal of a statistical significance test is to see if there is a statistically significant difference in mean standardised transaction amount between regular (class = 0) and fraudulent (class = 1) credit cards. Simply put, the observed values are distinct, genuine, and not the result of variability or chance. Here we have performed the t-test on two independent sample of two subsets of the data. The t-test is a common statistical method for comparing means, and its results provide insights into the potential significance of differences between the two classes.

5.2.8 Statistical Significance Test Result Analysis

The p-value obtained after performing a statistical significance test is a crucial metric in hypothesis testing. In our case the P-value obtained after performing **statistical significance test** was **0.0020895802939985045** which is very less than 0.05.

Thus it means that :-

- The p-value is less than the significance level (0.05), suggesting that there is sufficient evidence to reject the null hypothesis.
- This implies that the observed results are unlikely to have occurred by random chance alone.

CONCLUSION & FUTURE SCOPES

6.1 Conclusion

Without a doubt, credit card fraud is a crime, and cases of fraudulent transactions involving credit cards are on the rise globally. In this report, we attempted to develop models in order to distinguish between fraudulent and non-fraudulent transactions from a dataset obtained from Kaggle. After reading through a number of publications, we decided to employ six algorithms—Logistic Regression, Support Vector Machine, Random Forest, Neural Network, KNN & Decision Tree—to create our models. After successfully running the models, various results are achieved. According to the results of the experiments, the Random Forest Classifier had the highest validated F1 Score 0.998918785026693 & highest Accuracy of 99.96%, After the comparison of the three models, Statistical Significance Test was done & the P-value obtained was 0.00208958029399985045 which rejected the null hypothesis & implied that the observed results were unlikely to have occurred by chance. Other than that the logistic regression model's accuracy was the lowest among the three. Thus we can conclude that use of supervised Machine Learning algorithms in credit card fraud detection gives fruitful results.

6.2 Future Scopes

This project allows for the integration of multiple algorithms (*Naive Bayes*, *K-Means Clustering*) as modules, and the results of these can be combined to get a higher accuracy, which makes it more versatile. However, in order to obtain more detailed and real-time data, official support from banks is required. For that reason, we have developed methods to enhance the explainability & interpretability of the model's decisions. By improving the explainability & interpretability, we can gain the desired level of accuracy. banks, users, stakeholders. The Random Forest Classifier, which yielded the best accuracy in our research, may be used or incorporated into an application in the future to enable users to independently verify transactions to be fraudulent.

References

- [1] The dataset that is used in the report is taken from *Kaggle, Anonymized credit card transactions*, 2017.
- [2] Clifton Phua, Vincent Lee - *A Comprehensive Survey of Data Mining-based Fraud Detection Research*, Cornell University, 2010.
- [3] *Survey Paper on Credit Card Fraud Detection* by Suman , Research Scholar, GJUS&T Hisar HCE, Sonapat published by International Journal of Advanced Research in Computer Engineering & Technology (IJARCET) Volume 3 Issue 3, March 2014
- [4] Vaishnavi Nath *Credit Card Fraud Detection using Machine Learning Algorithms*, ICRTAC 2019. VIT.
- [5] Debachudamani Prusti and Santhnu Kumar Rath *Fraudulent Transaction Detection in credit card*, NIT Rourkella , 2019
- [6] Ryman-Tubb and Paul Krause *Neural Network Rule Extraction to Detect Credit Card Fraud*, University of Surrey, IFIPAICT, Volume - 363, 2011.
- [7] Ghosh, S., Reilly, D.L.: *Credit card fraud detection with a neural-network*. In: Proceedings of the Twenty-Seventh Hawaii International Conference on System Sciences, 1994, vol. 3, pp. 621–630. IEEE (1994)

CHAPTER 7

Biographical sketch

Sahil Dey

Melarmath, Agartala, Tripura PIN-799001,

E-Mail: deysagar3001@gmail.com, Contact. No. +91-9863082229

- Pursuing B.Tech. in Computer Sc. & Engg. branch from N.I.T, Agartala with CGPA of 8.21.
- Graduated High School from Umakanta Academy English Medium School under (T.B.S.E), Tripura with 76 % in 2017.
- Graduated Higher Secondary School from Shishu Bihar H.S School(T.B.S.E), Tripura with 81% in 2019.

Shubhadeep Nath

Dhaleshwar, Agartala, Tripura PIN-799007,
E-Mail: shubhadpn1927@gmail.com, Contact. No. +91-7005190502

- Pursuing B.Tech. in Computer Sc. & Engg. branch from N.I.T, Agartala with CGPA of 8.38.
- Graduated High School from Shishu Bihar H.S School(T.B.S.E), Agartala with 87.57% in 2017.
- Graduated Higher Secondary School from Shishu Bihar H.S School(T.B.S.E), Tripura with 87.4% in 2019.

Tithi Majumder

Ker Chowmani, Agartala, Tripura PIN-799001
E-Mail: tithimajumder717@gmail.com ,Contact. No. +91-8787672956

- Pursuing B.Tech. in Computer Sc. & Engg. branch from N.I.T, Agartala with CGPA of 8.60.
- Graduated High School from Shishu Bihar H.S School(T.B.S.E), Agartala with 84% in 2017.
- Graduated Higher Secondary School from Shishu Bihar H.S School(T.B.S.E), Tripura with 81% in 2019.

Sourajita Chanda

Joynagar, Agartala, West Tripura, PIN:799001, E-Mail:sourajitachanda8@gmail.com, Contact. No. +91-7005813692

- Pursuing B.Tech. in Computer Sc. & Engg. branch from N.I.T, Agartala with CPI of 8.6.
- Graduated High School from Shishu Bihar H.S School(T.B.S.E), Agartala with 81.14% in 2017.
- Graduated Higher Secondary School from Shishu Bihar H.S School(T.B.S.E), Tripura with 75.8% in 2019.